

American International University-Bangladesh (AIUB)
Faculty of Science and Technology

An Empirical Study on Static Testing Practices: Bangladesh Perspectives

Mojibul Hoque Jihad (21-45954-3)
Md.Tanvir Ahamed (21-45291-2)
Assadulla Ill Galib (21-45934-3)
Sharmin Jahan (21-45869-3)

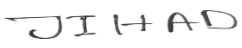
A Thesis submitted for the degree of Bachelor of Science (BSc)in
Computer Science and Engineering (CSE) at
American International University Bangladesh (AIUB
Faculty of Science and Technology (FST)

Fall 2025-2026 Semester
Submission Date: 28-07-2026

Declaration

This thesis is composed of our original work, and contains no material previously published or writtenby another person except where due reference has been made in the text. We have clearly stated the contribution of others to our thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financialand any other original research work used or reported in our thesis. The content of our thesis is the result of work we have carried out since the commencement of 'An Empirical Study on Static Testing Practices: Bangladesh Perspectives'

We acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate we have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors forany jointly authored works included in the thesis.



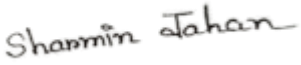
Mojibul Hoque Jihad
21-45954-3
Department of Computer science



Md. Tanvir Ahamed
21-45291-2
Department of Computer science



Asadullah Ell Galib
21-45934-3
Department of Computer Science



Sharmin Jahan
21-45869-3
Department of Computer Science

Approval

The thesis titled **"An Empirical Study on Static Testing Practices: Bangladesh Perspectives"** has been submitted to the following respected members of the board of examiners of the department of computer science in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science on **(28-07-2026)** and has been accepted as satisfactory.

MD. Anwarul Kabir
Associate professor & Supervisor
Department of Computer Science
American International University-Bangladesh

Saikat Baul
Lecturer & External
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Muhammad Firoz Mridha
Professor & Head (UG)
Department of Computer Science
American International University-Bangladesh

Mashiour Rahman
Senior Associate Professor & Faculty
Faculty of Science and Technology
American International University-Bangladesh

Prof. Dr. Dip Nandi
Professor & Dean
Faculty of Science and Technology
American International University-Bangladesh

Acknowledgement

First and foremost, we would like to express our sincere gratitude to our respected supervisor, Mr. Md. Anwarul Kabir, Associate Professor of American International University-Bangladesh (AIUB), for his continuous guidance, valuable suggestions, and unwavering support throughout the completion of our thesis. His insightful feedback and encouragement have greatly contributed to the successful completion of this work.

We are also deeply thankful to our external supervisor, Mr, Saikat Baul, Lecturer, for his constructive comments, guidance, and support, which helped us improve the quality of our research. We would like to extend our heartfelt appreciation to the faculty members of American International University-Bangladesh (AIUB) for providing us with the knowledge and academic foundation necessary to conduct this research.

Furthermore, we are grateful to all the participants who took part in our survey and contributed valuable data for this study. Without their cooperation, this research would not have been possible.

Finally, we would like to thank our family members and friends for their continuous encouragement, patience, and moral support throughout this journey.

This thesis is the result of the collective effort of our group members, whose dedication and teamwork made this work possible.

Table of Content

Declaration ……..2
Approval ……3
Acknowledgement ……..4
Table of Contents 5
Abstract 6
Chapter 1: Introduction 7
1.1 Introduction 7
1.2 Background Study 7
1.3 Motivation 8
1.4 Objectives 9
1.5 Scope of the Study 9
1.6 Findings 9
Chapter 2: Research Methodology 11
2.1 Introduction 11
2.2 Research Methodology 11
2.3 Literature Review 12
2.4 Empirical Survey 13
2.5 Summary 14
Chapter 3: Literature Review 15
3.1 Introduction 15
3.2 Literature Review 15
3.3 Research Gap 19
3.4 Our Survey 20
3.5 Summary 21
Chapter 4: Survey Analysis, Findings and Recommendations 22
4.1 Introduction 22
4.2 Analysis 22
4.3 Findings and Major Problems Identified 38
4.4 Recommendations 39
Chapter 5: Conclusion 41
5.1 Introduction 41
5.2 Research Outcome and Recommendation in Brief 41
5.3 Constraints 42
5.4 Possible Future Work 42
References 44
Appendix A……46
Appendix B……49

Abstract

Static testing is a key software quality assurance practice that identifies defects in software artifacts without executing the code. This study investigates the current state of static testing practices in Bangladesh, focusing on their awareness, adoption, implementation, and impact on software quality. It also examines the role of static testing in reducing defects through early verification activities such as reviews, walkthroughs, and inspections.

The research employed both literature review and empirical survey methods. A structured questionnaire containing 30 questions was used to collect data from software development and testing professionals in Bangladesh. The responses were analyzed to assess awareness, usage, perceived benefits, challenges, and organizational support for static testing.

The findings indicate that most participants recognize the importance of static testing in detecting defects in requirements, design documents, and source code before execution. However, its implementation remains inconsistent due to the absence of formal review processes, limited training, inadequate documentation, insufficient use of standardized checklists, time constraints, and weak management support. Respondents also identified limited automation and incomplete requirements documentation as significant challenges.

The study concludes that static testing can substantially improve software quality assurance in Bangladesh if implemented systematically. Organizations should integrate static testing into the software development life cycle, provide reviewer training, adopt formal review procedures and checklists, and strengthen organizational support. The findings contribute to a better understanding of current static testing practices in Bangladesh and provide practical recommendations for improving software quality, process maturity, and defect prevention.

Keywords:

static testing, software quality assurance, software reviews, defect detection, Bangladesh, survey analysis, software testing, quality improvement

Chapter 1

Introduction

1.1 Introduction

The use of software is an increasing necessity in contemporary society. It helps in supporting such key sectors as finance, healthcare, telecommunications, government administration, education, and e-commerce. In the digital economy of the present day, software systems are relied upon by businesses to operate their businesses, to deliver services, and be competitive. The larger, more complex and more interrelated software systems are, the harder it becomes to ensure they are correct, dependable, safe, and simple to maintain. Even minuscule bugs in code can lead to large issues in operations, loss of money, harm to your reputation, or security breaches. How to cope with bugs is one of the most significant issues in software engineering. Early defects in the Software Development Life Cycle (SDLC), during requirements analysis and system design may remain undetected until later in the life cycle, such as system deployment or system testing. Most real-life experiments have shown that the cost of correcting bugs increases significantly with their late detection during the development process [1, 2]. Therefore, it is very crucial to identify and correct issues at the early stages of the project in order to reduce project costs and increase the overall quality. Software Quality Assurance (SQA) provides systematic approaches to ensure that software products meet specified criteria and quality standards. One of the most significant quality assurance techniques as it assists in avoiding issues is called static testing. The static testing examines software artifacts like requirement documents, design documents, architecture diagrams and source code without executing the software [3]. By detecting ambiguities, inconsistencies, logical errors, incomplete requirements, and violation of coding standards, early in its development, the process of software testing dramatically reduces the amount of defect propagation across lifecycle phases. Modern software development practices such as Agile and DevOps incorporate the shift-left technique in a large part. It enhances testing and verification activities to begin as soon as possible [4]. Such an attitude is very close to the static testing where it is possible to continue with quality checks prior to the beginning of the execution-based testing. Although the use of static testing has been seen as effective, the manner in which the concept has been applied by various companies is very different, particularly in the growing software industries such as Bangladesh.

1.2 Background Study

A type of checking is Software Quality Assurance that checks the software under development or used in the production line. Mostly it is used to prevent occurrence of bugs rather than detection of bugs during execution. Dynamic testing A dynamic test of software executes the software with test cases in order to observe the behavior of that software. Static testing A static test of a computer program is a test that is run on the computer but does not execute the actual program. Its primary objective is to detect such bugs at the earliest stage possible in the development process such that the risks and cost of fixing them later in the development process is lesser. There are two main types of static testing: manual and automated. Requirement reviews, walkthroughs, technical inspections, peer code reviews, and document audits are all

parts of manual static testing. These methods entail looking at work in a systematic way. Stakeholders or domain experts verify the goods against issues, in a clear, complete, logically consistent, traceable, and standard conforming manner. Studies show that organized inspection techniques can get rid of a large number of errors before dynamic testing starts. This enhances efficiency in the development and reduced the efforts required to maintain it [2, 5]. As software engineering tools and automation technologies improve, more and more people are using automated static analysis as software engineering tools. Coding errors, security vulnerabilities, performance issues, and maintainability issues are identified with the help of rule-based checking, control flow analysis, data flow analysis, and pattern recognition. To make sure that code quality is always checked during the development process, these technologies are commonly built into Continuous Integration/Continuous Deployment (CI/CD) pipelines. Static testing is known around the world as an important part of high-maturity software development techniques. In other international standards, such as ISO/IEC/IEEE 29119 and others, attention is paid to systematic test documentation and verification operations [3]. However, in developing countries, such as Bangladesh, application of formal static testing methods is yet to be evenly distributed. Most of the firms emphasized more on dynamic testing due to pressure to produce, client expectations, and limited resources. This background is of much significance to know so as to look at the current situation of a stagnant testing in the Bangladeshi software sector.

Motivation and Objectives

1.3 Motivation

The rationale behind this study is the fact that the software industry in Bangladesh has been growing rapidly, and that there is a growing need to have high-quality, reliable, and secure software systems. Bangladesh has become a competitive outsourcing destination and most of the local companies now cater to international customers. The impact of software defects will be more serious as the complexity of the project is growing, and the project will experience cost overruns, missed deadlines, client dissatisfaction, and, possibly, loss of business opportunities. Though the idea of static testing is globally accepted as an effective tool in the detection of defects early in the design phase and in quality enhancement, its application in Bangladesh seems to be spotty. Many organizations are more interested in dynamic testing based on execution with little emphasis on the formal requirement reviews, formal inspections and automated static analysis tools. Also, the amount of empirical research, which systematically analyzes the current state of the practice of static testing in Bangladesh, is limited. The other driving force is the growing value of adhering to international quality standards. Global customers usually demand that they are provided with standardized quality assurance models. Unless preventive methods like static testing are appropriately implemented, organizations may not be able to attain a higher level of process maturity.

1.4 Objectives

This study aims to investigate the current practices of static testing in Bangladesh through a comprehensive survey and to evaluate their effectiveness in software development organizations. Based on the findings, the study identifies existing weaknesses and implementation gaps and provides recommendations to enhance the effectiveness of static testing practices.

The study seeks to achieve the following objectives:

- To examine how static testing activities are integrated into different phases of the Software Development Life Cycle (SDLC), including the commonly used static testing methods, tools, and review techniques.
- To evaluate the perceived effectiveness of static testing in reducing software defects and improving software quality.
- To identify the organizational challenges and barriers that hinder the effective implementation of static testing practices.
- To propose recommendations for improving the adoption and implementation of static testing practices in the software industry of Bangladesh.

1.5 Scope of the Study

The present study specifically concentrates on the practice of static testing in software development organizations that are in operation in Bangladesh. It discusses the activities of software testing performed in requirements analysis, system design and coding stages of Software Development Life Cycle. The research scope includes both manual techniques (including inspections and peer reviews) and automated tools of the static analysis. The research is founded on the results of survey conducted on software professionals in the form of: developers, quality assurance engineers, requirement analysts, software architects, and project managers. The study examines the level of awareness, rate of use, methodologies used, integration of tools, perceived effectiveness, and organizational issues associated with the implementation of the testing of statistics. It is not the primary concern of the study to use such a dynamic testing techniques as unit testing, integration testing, system testing, performance testing, or user acceptance testing. These techniques are mentioned merely to give contextual knowledge of how execution-based testing activities are complemented by static testing techniques.

1.6 Findings

The results of the present study indicate that an appreciable segment of the software professionals in Bangladesh is aware of the concept of static testing. Peer code reviews and requirement reviews are some of the practices that most respondents are familiar with. Nevertheless, the extent of formal and organized implementation is quite diverse among organizations. Formal inspection models are less popular compared to their manual review practices. Companies engage in informal consultations or reviews by their peers rather than subjecting their companies to regular standardized inspection processes. Automated tools of static analysis have not always been part of the development process, especially in small and medium-sized businesses. The factors that seem to affect tool adoption include project budget, organizational maturity, and client requirements. Based on the responses to the questionnaires, it was found that the main obstacles to successful implementation of the impact of the implemented strategies are the absence of formal training, management support, financial

resources, strict project deadlines, and standardized review policies. Although such issues exist, respondents usually confirm that early detection of defects enhances the quality of software and lowers the long term maintenance costs [1, 4]. As a whole, the study points to an apparent disconnect between internationally recommended best practices and their real-world application in the Bangladeshi software industry. There are ways to improve the quality assurance practices in the Bangladesh software quality assurance industry: strengthening training programs, promoting systematic inspection processes, and encouraging the broader use of automated tools of the static analysis industry.

Chapter 2

Research Methodology

2.1 Introduction

In this chapter, the research methodology that will be used in conducting the study on the practice of static testing in software engineering in Bangladesh will be presented. This research is aimed at investigating the existing situation in the sphere of the practice of the early detection of defects in the software development process, as well as proposing the recommendations, which can help to enhance the quality of assuring the software quality through the techniques of early defects detection. Because the concept of static testing is closely related to software reviews, inspections, walkthroughs, checklist-based verification, and document evaluation, the study needed a theoretical background based on the previous literature and a practical background based on the experiences of industry practitioners. In order to accomplish this goal, the study was planned to integrate the literature review and empirical survey techniques. The conceptual and contextual basis of the static testing, software quality assurance, and industrial quality practices were based on the literature review. The empirical survey assisted in gathering first hand responses of software practitioners to their familiarity with static testing, the level of its use within organizations, the prevalent limitations to its adoption, and the possible strategies to improve the use of the tool. Also, a systematic checklist framework was created of a face-to-face interview element to aid in further qualitative research of industrial practices. Thus this chapter describes the research strategy, data sources, study design and data collection procedures that were used in this thesis.

2.2 Research Methodology

This research was based on a mixed methodology, mostly qualitative in its interpretation, but backed by quantitative survey data. The research methodology has been chosen on the basis that the topic of static testing practice in Bangladesh needs both conceptual analysis and real-life experiences of the practice by its practitioners. In Bangladesh, past research on the quality assurance of software developed also utilized the empirical investigation through the survey to understand the industrial practices and challenges [1], [2].

In the course of the study, a comprehensive examination of the existing academic papers, journal articles, and conference publications has been conducted to comprehend the role played by a static testing in the field of software engineering. The studies reviewed included the foundations of software testing, assuring the quality of the software, quality control via inspection, tools of software analysis, and software engineering practices in Bangladesh [1]-[7]. The literature assisted in determining the key dimensions of the study including awareness of the existence of the early defect testing, the organizational adoption, techniques commonly used, barriers to implementation and the perceived benefits of early detection of defects.

Once a research problem was defined through literature, an empirical survey was developed, which would gather primary data by means of software practitioners. The choice to use a survey as a method to gather opinions has to do with the fact that a survey is a relatively efficient way of collecting structured opinions of multiple respondents. It is also in line with the previous studies on the quality assurance practices of software quality in Bangladesh,

which have already been conducted using questionnaires and organizational participation [1], [2].

The steps of the methodology of the study were as follows:

1. Determine the research problem and objectives;

2. Read the literature on the topic of the analysis of the quality assurance of software and static testing

3. Create a questionnaire using the results of literature review;

4. Publicize the questionnaire by software practitioners;

5. Gather and tabulate the answers;

6. Analyze the data using descriptive interpretation;

7. Find the major issues and suggest ways to address them.

The methodology of this thesis is then based on both secondary data of published literature and on primary data of the empirical survey.

2.3 Literature Review

The literature review aimed to provide a theoretical basis of the study and to determine research gaps in respect to the practice of static testing, particularly in the Bangladeshi software industry. The most common idea of statical testing is taking the form of analyzing software artifacts without the need to run the code. It involves reviews, walkthroughs, inspection, document verification and static analysis [5], [6]. The importance of these practices is that they assist in detecting defects at the earlier stages of software development and thus reducing the cost, rework and leakage of defects to later stages of software development testing [4], [6].

General literature in software testing reveals that software quality assurance is not solely confined to dynamic execution-based testing; but, in fact, it also extends to preventive practices, which, in fact, start as early as the requirements and design phase [3], [5]. Some of the earliest and most cost-effective methods of identifying ambiguities, inconsistencies, omissions, and design-level defects prior to the implementation process going any further are considered to be the static testing. The research on the analysis of quality at rest in software firms further indicates that the quality analysis and review based quality activities can be introduced with the relatively low effort and can be of great benefit to the currently established quality assurance methods [4].

A number of works additionally suggest that static testing can be performed either manually or with the assistance of the tools. Peer review, formal inspection, checklist-based document review and technical walkthrough are all manual methods. Some of the tool-assisted methods are automated static analysis, detection of bug patterns, code clone detection and architecture conformance analysis [4], [5]. Comparative research on the dynamic analysis tools reveals that the dynamic tools are relevant in determining the potential quality problems though selection of tools, false positives and interpretation of results are practical issues [7]. Static testing is also useful in security-oriented development to ensure the vulnerabilities are detected earlier in the SDLC before runtime testing is under-way [8].

According to Bangladesh perspective, previous studies have shown that software companies have appreciated the need to ensure that software quality is properly institutionalized, but many organizations continue to be challenged by constraints to institutionalize software quality properly [1], [2]. The Bhuiyan et al. survey revealed the current SQAT practices and

issues in Bangladeshi software companies and identified the need of an improved quality culture, structured practices, and enhanced testing practices [1]. On the same note, Sultana et al. noted that, due to time constraints, financial constraints, and immature processes, many companies (particularly small and mid-sized companies) do not fully adhere to standard SQA practices [2]. Another aspect in their work was the importance of improving and enhancing the quality of their work through inspection [2].

The recent discourse on software engineering practice in Bangladesh also reflects on the broader process gaps such as the inconsistent adoption of modern engineering practice and the unequal alignment with the global standards [9]. The findings apply to the practice of static testing since effective practices of static reviewing requires process maturity, the quality of documentation, experienced reviewers, and management commitment.

According to the reviewed literature, it can be concluded that the theoretical background of the use of statical testing can be considered to be well established as a preventive quality assurance method, yet its practical implementation in Bangladesh still seems to be rather limited and uneven. It is this lack of theory and practice which informed the current research and informed the creation of the survey questionnaire.

2.4 Empirical Survey

This study used an empirical survey as the primary primary data collection tool to investigate the true situation of the existing practice of the static testing in Bangladesh. The survey was to be used to obtain knowledge of the practitioners, experience in their organization, as well as perception of the practitioners towards the subject of static testing in the software development.

The questionnaire survey questionnaire was prepared on the basis of the significant themes found during the literature review. These themes included:

• Knowledge and experience of static testing use of reviews, inspections and walkthroughs.

• Organizational assimilation of the activities of the static testing.

• The tools and techniques employed in detection of defects early in the manufacturing process.

• Problems with using the static testing.

• Effects of a non-executing test on the quality of the software.

• Suggestions of how to better the existing practices.

The questionnaire contained close ended questions and opinion based questions, which made the study to not only obtain structured responses, but also practical suggestions by the participants. The responses were gathered among the software-related participants involved in software engineering activities. The data obtained were analyzed to draw patterns, weaknesses, and opportunities associated with the practice of static testing.

The empirical survey method suited this study since it helped the researcher to reduce systematic evidence of the industry in a systematic way. Moreover, effective methods based on surveys have already been utilized in previous investigations of the software quality assurance practices in Bangladesh [1], [2]. The empirical survey in this thesis therefore

becomes the linkage between theoretical results of literature, and the practical realities that respondents are subjected to.

2.5 Summary

This chapter explained the research methodology that the research adopted. The study had a mixed methodological orientation in that it integrated literature review and empirical survey. The literature review assisted in establishing the conceptual frame of the notions of static testing, software quality assurance, and the context of Bangladesh software industry. The empirical survey offered first-order evidence in terms of current practices, challenges, as well as opportunities in relation to the practice of static testing. Moreover, a structured interview checklist was elaborated to facilitate further qualitative study of the experiences of practitioners. Combined, these approaches allowed an appropriate basis to analyze static testing practices in Bangladesh and create findings and recommendations in the following chapter.

Chapter 3

Literature Review

3.1 Introduction

Software quality assurance and testing are key components of software engineering since they help to identify defects, enhance reliability, and minimize the risk of failure post-deployment. Early identification of defects is particularly important since errors that are detected during the initial stages of development are not only simpler and less expensive to fix than post-release defects. In this respect, a major role is played by the static testing since it is a method that evaluates the static artifacts of the software without the necessity of running the program. Common methods of performing static testing are requirement reviews, design reviews, inspections, walkthroughs, code reviews, and static analysis of source code. The larger literature demonstrates that software testing could be considered in various categories. General review studies define testing as a fundamental activity of the SDLC and differentiate between static/manual and dynamic/automated methods of testing as well as outline testing levels including unit, integration, system, and acceptance testing. These researches are helpful in conceptual grounding, however, it does not directly deal with the practical status of the static testing in Bangladesh. The literature available in Bangladesh has largely looked at the quality assurance of software and software testing in general and not in specific terms of static testing. One of the most important local studies is the 2018 national survey by Bhuiyan et al. which examined SQAT practices and challenges in 47 surveyed organizations and highlighted organizational characteristics, testing practices, teams, tools, and challenges. Later research by Sultana et al. (2020) also demonstrated that over fifty percent of the surveyed companies in Bangladesh fail to comply with SQA practices throughout the SDLC, and proposed an improvement model based on inspection. Since this thesis is concerned with the practices of statical testing in Bangladesh, this chapter gives the review of the directly related studies as well as the supporting international studies. This is to determine what is already known, what is still requiring exploration and where this research fits in.

3.2 Literature Review

The literature reviewed can be divided into three general areas:

(1) concepts of general testing and statical testing,

(2) empirical research on the tools of static analysis and the methods of researching the static quality

(3) Studies on quality assurance and testing practices focused on Bangladesh.

3.2.1 General background on testing and static testing

The literature reviewed presents a number of papers that give general definitions and classifications of software testing. Such studies clarify that testing is undertaken to enhance

software correctness, quality and dependability and differentiate between the static/manual and dynamic/automated methods. They also outline the primary testing levels - unit, integration, system and acceptance testing - which can help in understanding how the testing activities are spread throughout the development lifecycle.

Even though these papers are not necessarily focused on Bangladesh or fixed testing practices in industry, they are still significant to this thesis as they lay the conceptual foundation. Specifically, they demonstrate that, in fact, static testing is not a stand-alone peripheral activity; instead, it is actually a subset of a larger quality assurance strategy, which is supposed to detect defects as early as possible.

Another related supporting paper on the topic of static testing and defect leakage also argues that, static testing can help prevent the spread of defects by analyzing artifacts prior to execution and by detecting problems in requirements and design early in the lifecycle. This helps in forming the general perception that the focus of the application of the static testing is prevention-oriented, and not necessarily corrective. Nevertheless, these papers are largely abstract and not a comprehensive picture of the organizational practice today in Bangladesh.

Hence, such general papers come in handy in defining terms and establishing theoretical background but indirectly connected with the current study.

3.2.2 Static analysis tools and static quality techniques

A more closely applicable set of literature discusses the tools of static analysis and their usefulness. According to Nikolić et al., source code can be improved with the help of static code analysis which identifies bugs, vulnerabilities, code smells, duplications, as well as deviation of the code of conduct. Their work also defines the concept of a code review cycle in which a tool is run, results are viewed and code is then refined. This is significant as it demonstrates that the idea of static testing should not be perceived as merely the output of the tool, but as a formalized activity that is related to human evaluation and reengineering.

The 2023 comparison of six tools of the static analysis is another important empirical contribution. A study conducted by Leonarduzzi et al. on 47 Java projects using Better Code Hub, CheckStyle, Coverity Scan, FindBugs, PMD and SonarQube revealed that there was minimal or no consensus among the tools. They further indicated that the precision of tools fell between 18% and 86% indicating that false positives are still a significant issue. This observation is of great relevance to the current thesis statement since it implies that the decision to adopt a static analysis tool does not necessarily mean that one can be sure of quality assessment. The output of the tools still needs to be carefully interpreted.

The 2024 research paper on secure code review offers a more recent and practical perspective of the static application security testing (SAST). The authors used 815 vulnerability-contributing commits across 92 C/C++ projects, and found that a single SAST could warn about at least one vulnerable function in 52% of commits, and combining tools increased that to 78%. They also demonstrated that prioritization at the time of code review based on warning indicators, enhanced accuracy by 12 percent, recall by 5.6 percent, and initial false alarm by 13 percent. Simultaneously, they discovered that at least 76 percent of the warnings in vulnerable functions were not relevant and 22 percent of the commits that contributed to vulnerability remained unnoticed. This paper shows the practicality and the ineffectiveness of the concept of the static analysis in real development environments.

A related 2021 paper by Joshy et al. suggested a hybrid approach known as Helium, changing any statical warnings into executable test fragments which can be tested accordingly. The authors evaluated 1,955 warnings of two commercial tools evaluated on 12 C projects and generated 1,003 test cases and identified true positives, false positives, and likely false positives. The significance of this work is that it deals with one of the main flaws of the tools in the statical approach: uncertainty in the validity of warnings. It indicates that hybrid solutions can enhance static testing through providing warnings that are easier to act upon.

The 2016 SME-focused study is also very relevant. It studied the introduction of automated static analysis in five small and medium-sized enterprises and found that the effort to introduce a technique of static analysis was generally low, mostly less than one person-hour per technique, and that there were multiple defects found in production code. The companies that took part found the analyses helpful and decided to include them in their QA process. This is particularly significant to Bangladesh where most companies are small or medium-sized and lack of resources is frequent.

Considered collectively, these international studies point out four significant things. To begin with, the quality of code and code review can be effectively supported by the use of static analysis. Second, there is no single tool that can be considered to be adequate fully due to the wide variation in detection capabilities as well as in precision. Third, the combination of static testing with the review practices or the complementary validation is more helpful. Fourth, even resource-constrained organizations can realize feasibility of using the techniques of static quality.

3.2.3 Bangladesh-focused studies on software quality assurance and testing

The top most directly related local investigation is the survey of 2018 by Bhuiyan et al. To learn about software quality assurance and testing practices and challenges in Bangladesh, the authors carried out a qualitative survey between August 2017 and January 2018 with 47 organizations, mostly based in Dhaka. The research took into account the characteristics of organizations, testing methods, teams, tools, challenges, training, education and career opportunities.

Their results provide a handy point of reference to this thesis. Over half of the sample of firms had less than 10 years of service, which is indicative of the fast pace of the ICT sector development in Bangladesh. Of the 47 organizations 37 had dedicated test teams and 10 did not, and the number of developers was typically larger than the number of testers. It was also indicated in the study that 68% of the respondents indicated that they follow constructed methodologies of testing all the time, but 9% of the respondents said that they do not follow any structured methodology at all.

Regarding QA activities, the most common activity performed was testing, which was followed by inspection/ formal review and walkthrough/ informal review. This is a significant point when it comes to the current thesis since inspection and walkthrough are core types of static testing. Simultaneously, the research concluded that functional testing was the one, which received the most attention, then there was regression testing, and comparatively less attention was given to automated testing. This implies that, despite the existence of review-based static activities, industry effort still skews towards more broad-scale testing work as opposed to more formally defined static testing practices.

The 2018 survey also gives insights into the use of tools. Most commonly mentioned tools were JIRA and Selenium, followed by Apache JMeter, Appium, and Redmine, and most tools used were open source. This implies that it is cost sensitive and prefers open-source ecosystems, and this may affect adoption decisions of the static testing tools, too.

The paper also established some challenges: it is important to assure the quality of software within the budget and time, it is important to ensure the complete coverage of the test, it is important to conduct early tests, it is important to understand the requirements, it is important to cope with the dissimilarity of the devices and conditions, it is important to test in Agile settings, and it is important to address the lack of organized systems and professional staff. The authors also noted that the tester-to-developer ratio was frequently around 1:2 and in some cases, could be as high as 1:10, and that 22% of firms engaged in testing, but this was often due to client requirements rather than internal quality commitment. These results indicate that there are structural flaws that can directly influence the routine implementation of the practice of static testing.

The second large study with a Bangladesh focus is the 2020 empirical study by Sultana et al. The survey of nine established software companies in Dhaka found more than half of them wanting in following SQA practices in the SDLC. The authors designed a cross-sectional survey involving 17 questions over 8 SQA parameters and targeted software professionals such as developers, testers, and project managers.

Their results add to the image that the survey of 2018 has made. The study has indicated that 61 percent of respondents agreed that projects have very limited time to develop quality software and 54 percent said they would not like to see a lot of project planning and process implementation. 73 percent of respondents agreed that coordination and support provided by developers is a core concern to SQA, 57 percent indicated lack of training and experience and 72 percent agreed that testing is often conducted by people who do not have adequate technical and procedural knowledge. These findings indicate the problem is not that there is no testing activity available, but rather that there is a lack of quality, discipline and organizational support surrounding the testing activities.

One of the most valuable contributions of the study in 2020 is the proposed Quality Inspection Model (QIM). The authors define QIM as a very lightweight, inspection-based model, which is simple to learn and adopt, can be blended into the SDLC with minimum additional resource requirements, and can be used to help find and fix faults when the system is still under development. They also note that organizations need to develop a culture of quality assurance, enhance communication between team members, and endeavor to organize dedicated SQA teams with sufficient expertise. This paper is very important to a thesis on the practice of static testing since it explicitly provides the path of improving practices in Bangladeshi firms through explicit presentation of the path of improvement in the form of inspection.

It is also pertinent to the 2013 Bangladesh paper on cost reduction. It suggests integrating a formal inspection with a tool of static analysis in order to identify defects during the earlier stages of development and argues that this combination of techniques can minimize rework, cost, and time through the earlier detection of defects. Although this work is more prescriptive and narrow in scope as compared to the 2018 and 2020 surveys, it holds significance in that it explicitly connects the concept of static analysis and inspection to the local software quality context.

3.2.4 Studies that are only partially related

Not every paper gathered could be equally pertinent to this thesis. There are other studies that can only be used as supplementary references.

An example of a technical relationship but whose central issue is maintenance and impact prediction, and not the organizational practice of software change impact analysis by using static analysis. It describes that the static methods take all possible program behaviors but usually contain an excessive number of false positive, whereas the hybrid methods attempt to achieve a better precision and recall. It is also handy as technical background, but not the subject of the present study.

Likewise, the article on the use of PMD in education applies to the interpretation of how people perceive static warnings, but it is focused on the classroom practice instead of industry or Bangladesh. The Pakistan research on static testing in software development in the developing country is also useful in demonstrating that the issue of static testing in the development of software in the developing country is by no means a foreign subject. General security-testing papers and general studies of software engineering in Bangladesh also do give some context, but do not specifically analyze the practice of static testing.

Thus, these works should be considered as supportive literature, not as the main evidence to support the thesis argument.

3.3 Research gap

Based on the literature reviewed, it is evident that there is a research gap.

To begin with, the 2018 and 2020 Bangladesh studies are general in nature, as they are concerned with the overall quality assurance and testing of software, but not specifically with the practice of static testing. They involve inspections, formal reviews and other activities but not isolated as the major object of study. Consequently, much is still unknown regarding how specifically the Bangladeshi firms comprehend, embrace and execute the use of the static testing methods like reviews, walkthroughs, inspections and the use of the static analysis tools.

Second, there are solid evidences of the strengths and weaknesses of the use of a static analysis in the international tool-oriented studies, but they fail to describe the Bangladesh-specific organizational context. They demonstrate tool disagreement, false positive, adoption effort, and benefits of code review, but do not describe how local companies with budget, skill, and process constraints apply or abandon static testing.

Third, in the Bangladesh literature, challenges that include limited time of the project, low maturity of the process, inadequate training, poor coordination, no dedicated project teams, and little automation effort are recurrently identified. Nonetheless, the amount of concentrated evidence about the impacts of these challenges on the adoption and effectiveness of static testing in particular is still lacking.

The thesis fills that gap by looking at the practice of static testing more squarely in the Bangladeshi software sector.

3.4 Our Survey

According to the literature review above, this thesis makes its survey an intensive study of the practice of static testing in Bangladesh. Compared to the previous studies on Bangladesh that considered software quality assurance and testing in general, the current survey focuses specifically on such practices as reviews, walkthroughs, inspections, code review and the use of a tool such as a static analyzer.

The literature indicates that already in Bangladeshi companies, there are already some examples of the use of the two most commonly used QA activities to date namely inspection/formal review and walkthrough/informal review. The 2020 paper further put forward a lightweight inspection-based model to organizations that cannot cope with structured SQA practises. Simultaneously, both studies showed that there were still persistent barriers in terms of limited time, poor planning, lack of skilled personnel, insufficient dedicated SQA teams, and inconsistent process maturity. Such observations suggest that a static testing can be defined in concept but not necessarily implemented systematically.

Thus, the survey of this thesis is aimed at delving deeper into the following areas:

• How the concept of the static testing is perceived by the software professionals in Bangladesh.

• What is the kind of static testing that is being practiced in organizations.how frequently is it reviewed, inspected, walkthroughs, and static analysis.

• What tools are embraced and what determines their choice of tools

• What advantages do practitioners believe they have when using a static test.

• What are the organizational, technical and human obstacles that inhibit adoption;

and how the practice of the statical testing is different in large, medium, and small firms, according to their maturity level, project environment, and the size of the firm.

By doing so, this current survey builds upon the past Bangladesh studies in two key aspects. To begin with, it makes the SQAT, in general, and the static testing, in particular, the subject of the investigation. Second, it relates the results in Bangladesh to the rest of the evidence available on the use of the tools of static analysis, false positives, code review integration, and lightweight adoption in resource-constrained environments. This renders the survey locally-based and internationally-informed.

Consequently, this thesis is likely to give a better understanding of whether the practice of static testing in Bangladesh is practised as a regular quality discipline, selectively applied under project pressure or perhaps it remains a secondary exercise vis-a-vis dynamic testing. The survey will also provide useful suggestions to the industry and academic stakeholders on training, process enhancement, and adoption of tools.

3.5 Summary

The chapter has surveyed the literature available on the practice of static testing in software engineering with specific emphasis on Bangladesh. The review demonstrates that software testing is a fundamental SDLC activity and that static testing is an important preventative practice by assisting organizations in identifying defects before they can be executed. International studies also indicate that because of their limited agreement, false positives, and incomplete detection, the use of the static analysis tools can be used to both support the code review and quality improvement processes.

Within the context of Bangladesh, the current literature shows that QA and testing practices are on the increase, although there are some critical flaws present. The 2018 survey reported variation in the team structure, methodology used, tool adoption, and the major challenges like time pressure, unskilled personnel, and poor practices of early testing. The 2020 study indicated that over fifty percent of the surveyed companies fail in the area of SQA adherence and suggested an inspection-based model as a viable way of improvement. The 2013 paper also emphasized the usefulness of the integration of a static analysis and formal inspection to minimize rework and cost.

Nevertheless, the studies reviewed are not yet able to offer a specialized analysis of the static testing practices in Bangladesh as a standalone issue. The gap thereof provides the current thesis survey. The second chapter is then a continuation of this literature to introduce the research methodology that was employed in this study.

Chapter 4

Survey Analysis, Findings and Recommendations

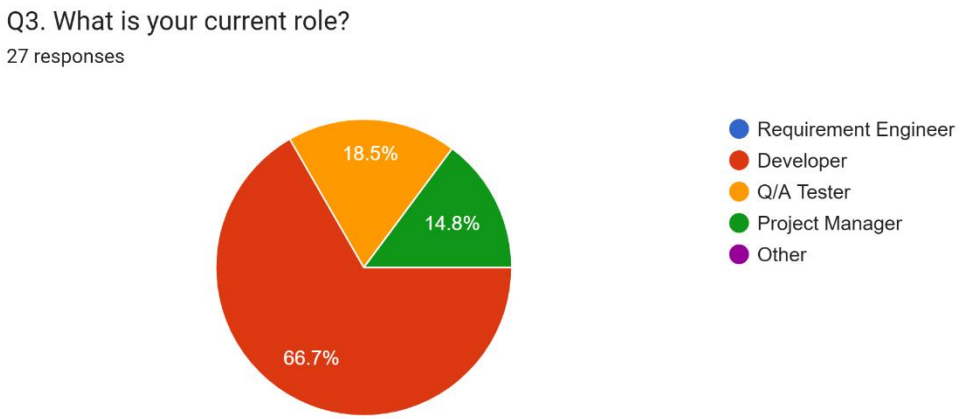
4.1 Introduction

The chapter gives the analysis of the survey done to the study on the practices of static testing in Bangladesh. This chapter aims at interpreting the responses obtained after interviewing the software professionals as well as linking the responses with the ultimate objectives of the study. There were 28 valid responses that were analyzed. The questionnaire had 31 questions that included respondent profile, knowledge of the existence of a test of this kind, practices within their organizations, tools that are used, advantages of the test and its effectiveness, necessity of the test and the design, review of the codes, measures, difficulties, and the possible improvement strategies. The chapter has been divided into three large sections. In section 4.2, the detailed analysis of the results of the surveys is presented in the form of statistical interpretation, comment-based observations, and discussion of the results. Section 4.3 is a summary of the key problems and the most important results found in the survey. In section 4.4, the recommendations are given depending on the identified gaps, challenges, and an opportunity to improve. Collectively, these sections present a clear image of the present state of the practice of static testing among the surveyed software professionals and organizations.

4.2 Analysis

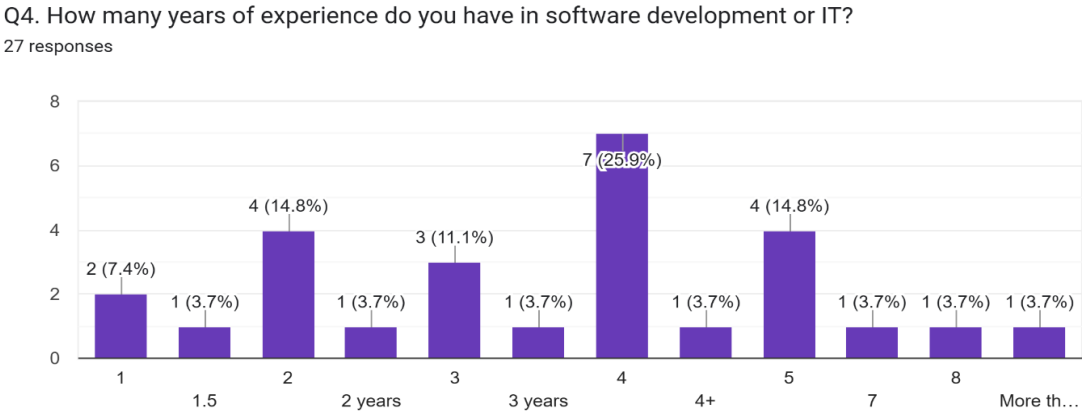
Q3. What is your current role?

The responses indicate that the survey respondents belonged largely to technical field and are directly involved in software development and quality activities. The Developers constituted the largest group with 18 respondents (66.7%). It was then followed by Q/A Testers having 5 respondents (18.5%) and Project Managers having 4 respondents (14.8%). There were no answers to Requirement Engineer or Other. This distribution shows that the survey results are strongly influenced by practitioners who are closely involved with the day-to-day software development and testing activities. Consequently the answers are apt to be based on working industry experience and not necessarily on managerial perceptions.



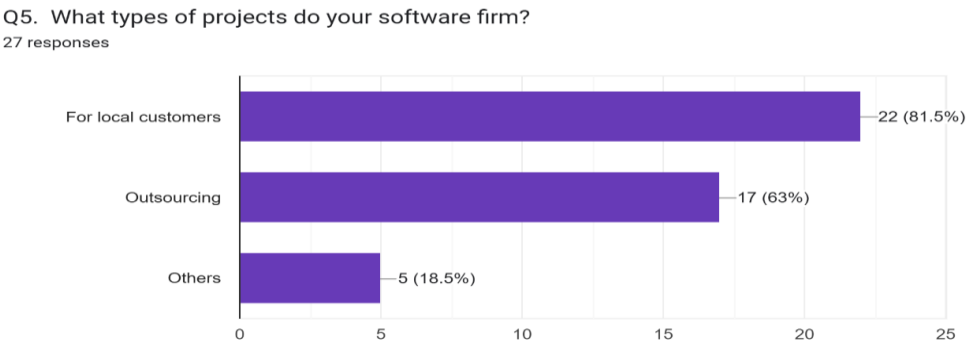
Q4. How many years of experience do you have in software development or IT?

The answers show that, the participants had diverse collection of professional experience, between 1 year and over 5 years. The highest single entry was 4 years, with 7 respondents, then 2 years and 5 years with quite remarkable representation. When similar items like 4+ are taken together, respondents who have around four years experience make the highest number. Less often numbers of participants reported 1 year, 1.5 years, 3 years, 7 years, 8 years, and over 5 years. This indicates that the sample is comprised of both early and mid-career professionals with a higher concentration of respondents who already have several years of industry exposure. Thus, the majority of the respondents seem to have enough practical experience to make informed views regarding the practice of static testing.



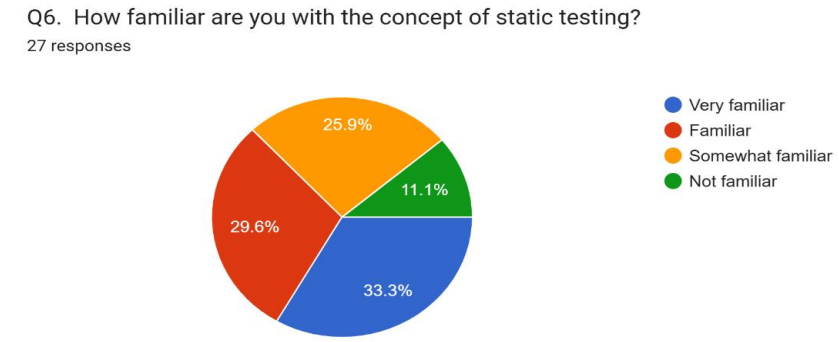
Q5. What types of projects do your software firm handle?

This was a question that is a multiple response. The results indicate that the sampled companies are engaged in different kinds of software projects. The choice of Local customers was made by 22 respondents (81.5%), Outsourcing was made by 17 respondents (63.0%), and others by 5 respondents (18.5%). The preeminence of localized customer projects implies that a majority of the involved companies are highly active in the local market. Meanwhile, the high amount of the outsourcing work suggests that lots of companies are also occupied with external driven or international client-driven development environments. As the percentages are above 100, it is clear that there are a number of firms which handle both local and outsourced projects simultaneously. Such a mixed project environment can have an impact on the acceptance and the rigor of the practices of static testing.



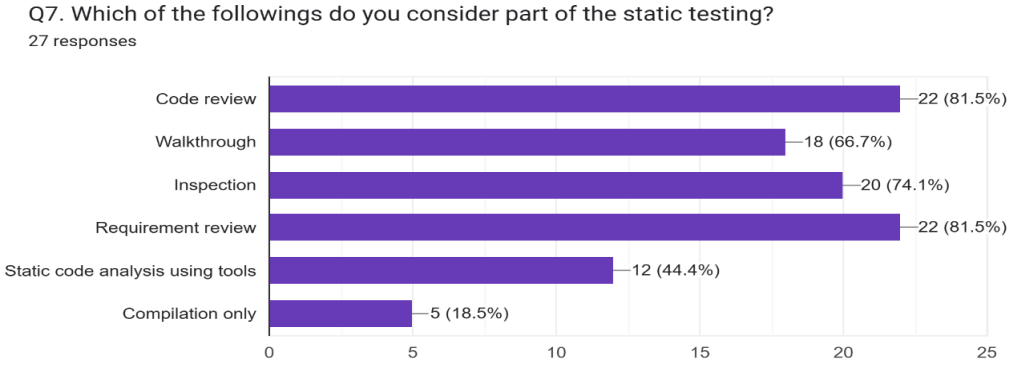
Q6. How familiar are you with the concept of static testing?

The responses show that the general level of awareness about the subject of static testing is good. Very familiar was chosen by 9 respondents (33.3%), Familiar by 8 respondents (29.6%), Somewhat familiar by 7 respondents (25.9%), and Not familiar by 3 respondents (11.1%). On the whole, over three-fifths of the respondents said they were either familiar or very familiar with static testing. This indicates that most of the participants had a good conceptual and overall conceptual knowledge of the topic, which reinforces the reliability of their subsequent responses regarding practice and effectiveness.



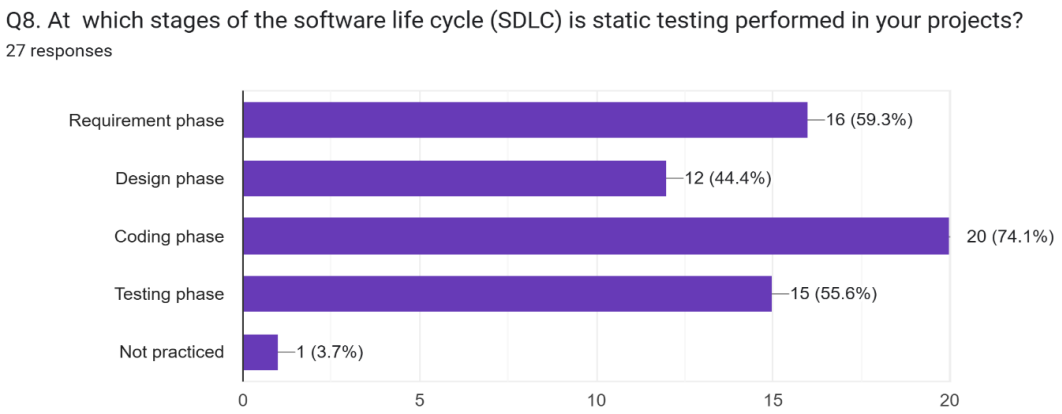
Q7. Which of the following do you consider part of static testing?

The item was a multiple response question that was used to determine conceptual knowledge of the static testing activities. Code review and Requirement review were selected by the highest number of respondents (22 respondents each, 81.5%). Inspection with 20 respondents (74.1%), Walkthrough with 18 respondents (66.7%), and Static code analysis using tools with 12 respondents (44.4%). Compilation only, in turn, was chosen by 5 respondents (18.5%). These results can imply that review-based activities are mostly recognized by the respondents as the key elements of the process of static testing. Nevertheless, the smaller percentage of tool-based static analysis and the fact that some of the respondents have indicated that the only thing they do is to compile further suggests the presence of some respondents with half baked misconceptions or who have not understood the entire scope of static testing.



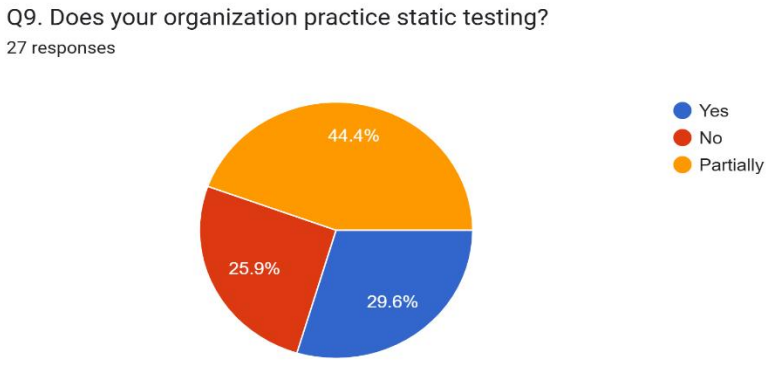
Q8. At which stages of the software life cycle (SDLC) is static testing performed in your projects?

The findings indicate that the practice of static testing is practiced at various phases of the SDLC, but not necessarily uniformly. The most frequently selected phases were the Coding phase with 20 respondents (74.1%), Requirement phase, 16 respondents (59.3%), Testing phase, 15 respondents (55.6%) and Design phase, 12 respondents (44.4%). It is only 1 respondent (3.7%) who said that there is no practice of static testing. This trend suggests that source code review and activities related to coding are the most closely linked activities with static testing. The lower percentages in the initial stages of requirements and design, though a moderate number of respondents also used static testing in the early stages of project development, suggest that the use of pre development static verification is not as firmly established as code level review is.



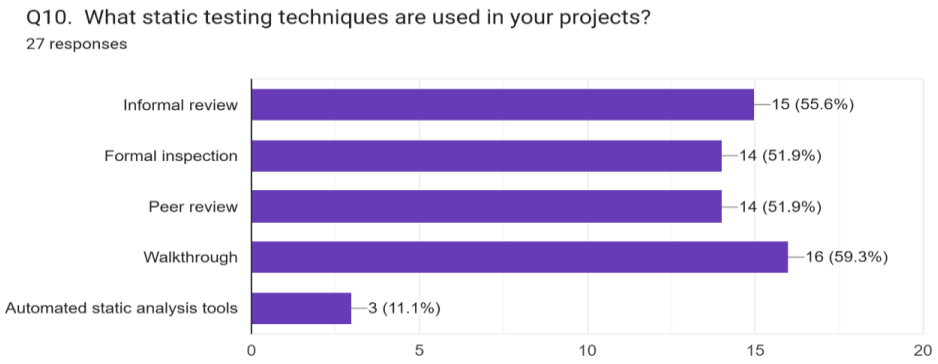
Q9. Does your organization practice static testing?

The responses paint a relative picture of organizational adoption. The most common response was partially, with 12 respondents (44.4%). Yes had 8 respondents (29.6) and No had 7 respondents (25.9) respectively. These results suggest that the complete institutionalization of the static testing is still restricted in most organizations. The fact that the largest group of those who chose to respond to the question partly, suggests that the concept of static testing is present in many firms, but is not consistently implemented, informally run, or applied to a specific project or phase of development.



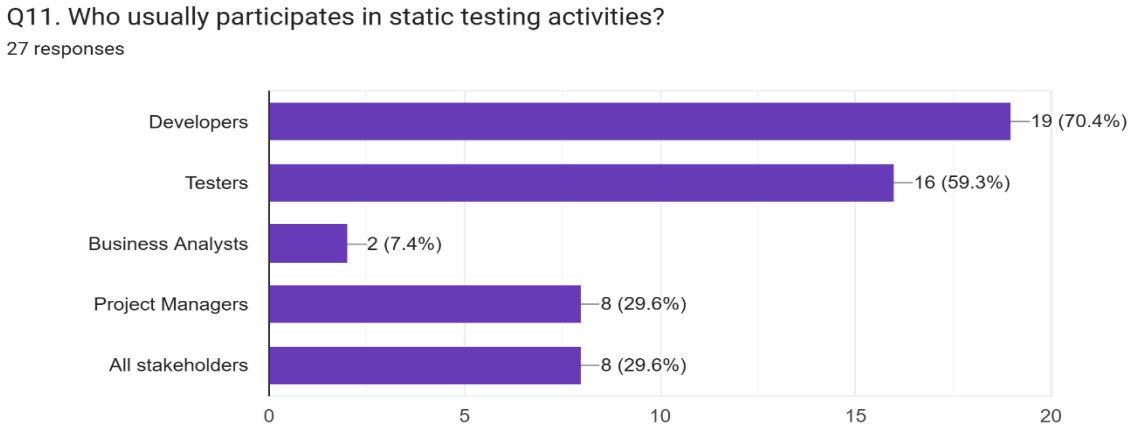
Q10. What static testing techniques are used in your projects?

This was a multiple respond question as well. Walkthrough, which was chosen by 16 respondents (59.3%), was the most commonly used technique. Formal inspection and Peer review were each chosen by 14 respondents (51.9%). 15 respondents (55.6%), selected informal review. On the contrary, only 3 respondents (11.1%), selected Automated static analysis tools. The findings indicate that the human-focused and review-based nature of the influence of the tool is predominant in these companies in terms of the statical testing. The informal reviews, walkthroughs, and peer reviews seem to be the most common ones, and automated analysis tools are relatively underutilized.



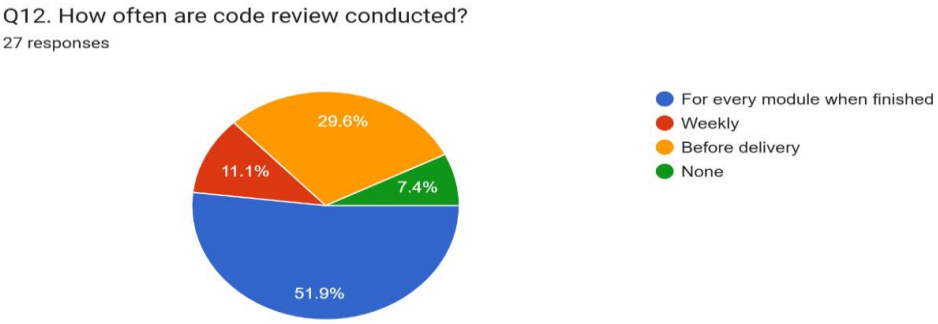
Q11. Who usually participates in static testing activities?

The findings show that static testing is mainly conducted by technical team members. 19 respondents (70.4%), and 16 respondents (59.3%), selected developers and testers respectively. Project Managers and All stakeholders were chosen by 8 respondents (29.6%), and 2 respondents (7.4%), respectively. It signifies that static testing is mostly regarded as the task of developers and testers. The reduced involvement of business analysts is quite significant, given the significance of requirement reviews in ensuring early fault detection. It can imply that the cross-functional participation in the static testing is still restricted in most organizations.



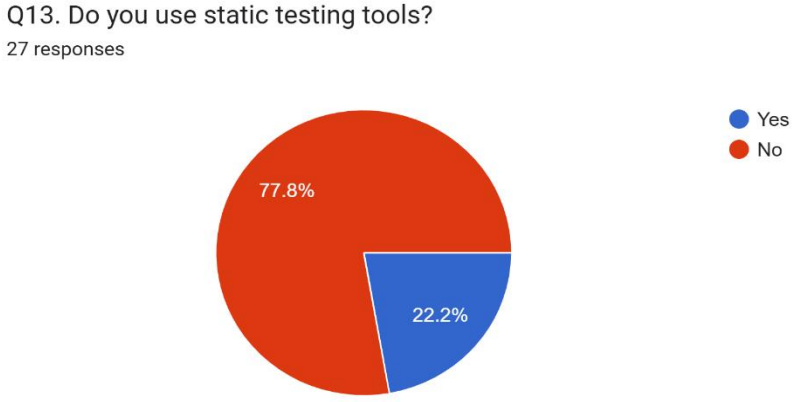
Q12. How often are code reviews conducted?

When answering the question For every module when finished, the most frequent choice was selected by 14 respondents (51.9%): This was then preceded by Before delivery with 8 respondents (29.6%), Weekly with 3 respondents (11.1%), and None with 2 respondents (7.4%). These findings indicate that code review is a common practice in numerous organizations but the time interval varies. Overwhelming dominance of module-based review suggests that reviews tend to be incorporated into development workflow. But the fact that some firms do late in the lifecycle or do not do reviews at all suggests that some firms may be diminishing the preventive value of the static testing.



Q13. Do you use static testing tools?

The responses indicate that the use of specific tools of conducting static tests is relatively low. The respondents who said Yes were only 6 (22.2), as against 21 (77.8) who said No. This clearly suggests that majority of the surveyed organizations still tend to adhere more to manual review practices than those automated static testing tools. The poor adoption of the tools can be linked to cost, their lack of expertise, poor organizational focus, or poor awareness of the available tools.

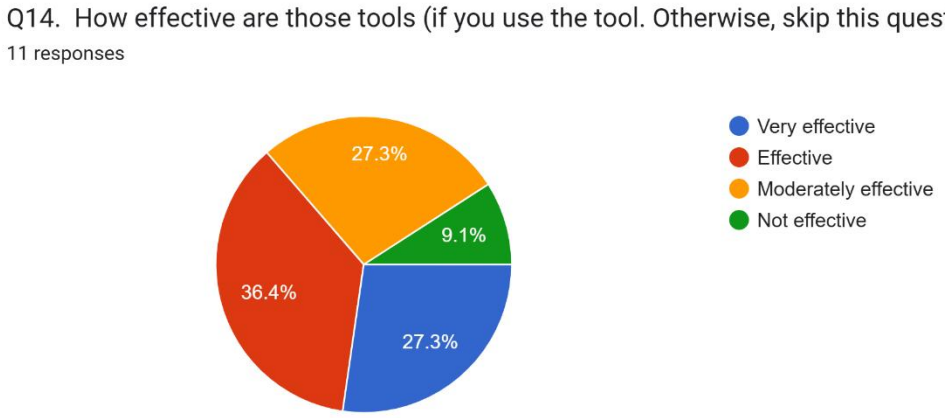


Some of the tools that respondents mentioned included sonarQube, ESLint, PMD, Checkstyle, GitHub Code Scanning, PHPStan, Prettier, TypeScript Compiler, GitHub Pull request Code Review, Lightrun, and SAST. Selenium, Appium, Postman, and JMeter are also more typically thought of as dynamic or automated testing, rather than a more static approach to testing. Such response pattern indicates that although some of the organizations do use known tools of testing which are static, there are also confusions among the respondents with regard to the difference

between the tools of testing, which are known as static and dynamic in testing. This observation is significant as it indicates a potential knowledge gap in practical tool classification.

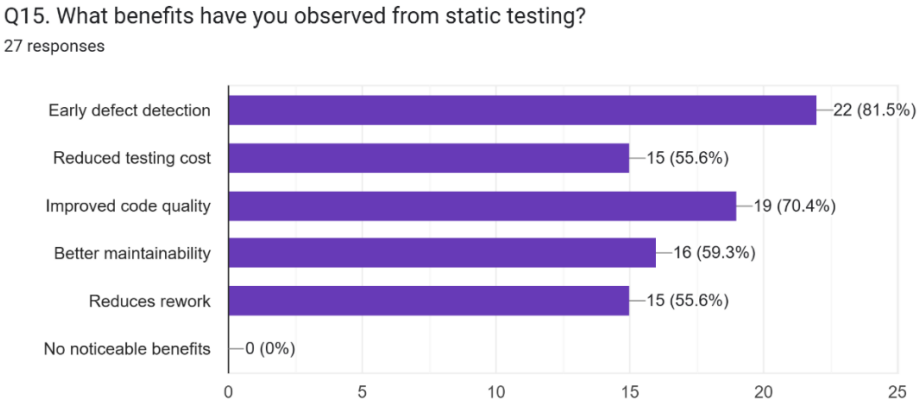
Q14. How effective are those tools?

This was an optional question that received 11 answers. Out of them, Effective was selected by 4 respondents (36.4%), Very effective by 3 respondents (27.3%), Moderately effective by 3 respondents (27.3%), and Not effective by 1 respondent (9.1%). The use of the tools seems to be less than expected, but the feedback provided by the people who tested the tools was mostly positive. The overall results indicated that the tools were at least moderately useful by most respondents, which implies that higher adoption of the tools of static testing may be of additional value in case the organizations are adequately trained and equipped to use the tools.



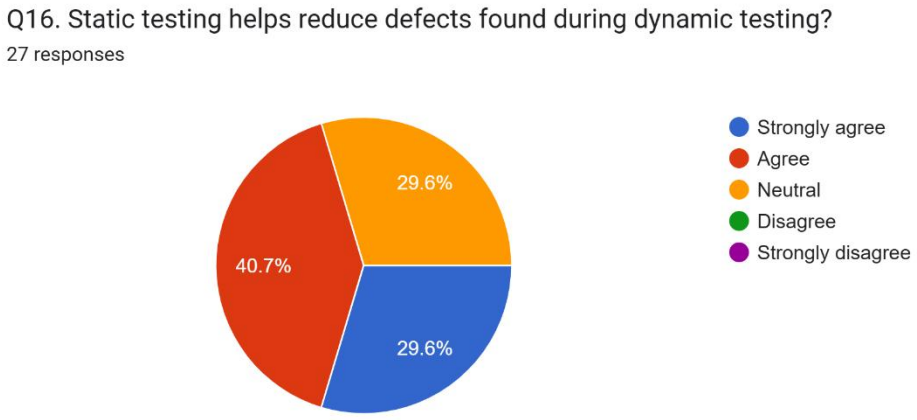
Q15. What benefits have you observed from static testing?

This question is a multiple response, and the answers are very much in agreement with the advantages of a static test. The most common identified benefit was Early defect detection, which 22 respondents (81.5%), chose. This was topped with Improved code quality with 19 respondents (70.4%), Better maintainability with 16 respondents (59.3%), and both Reduced testing cost and Reduces rework, each was selected by 15 respondents (55.6%). None of the respondents chose No noticeable benefits. These results go a long way in supporting the opinion that the use of static testing contributes to the practice of software development projects. Specifically, the fact that the emphasis is put on early defect identification implies that respondents view the concept of static testing as an activity of preventative quality assurance measure as opposed to a formal review activity.



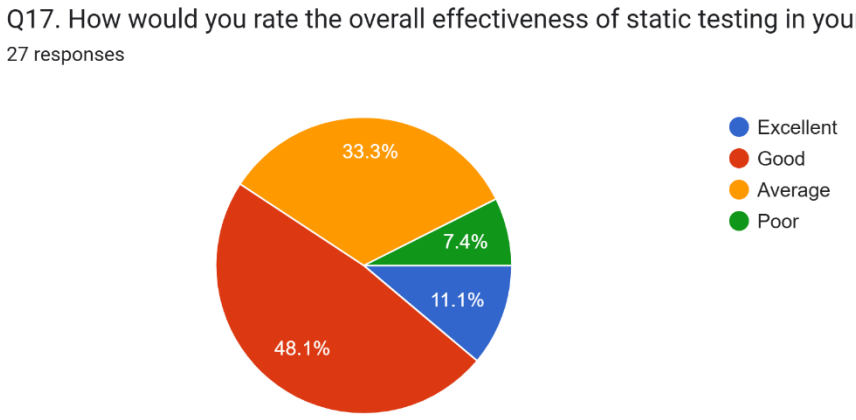
Q16. Static testing helps reduce defects found during dynamic testing.

The reactions were very favorable. Agree was chosen by 11 respondents (40.7%), Strongly agree by 8 respondents (29.6%), and Neutral by 8 respondents (29.6%). None of the respondents chose Disagree or Strongly disagree. This suggests that the surveyed professionals largely feel that the presence of a static testing would help in reducing the number of defects prior to the start of an execution-based testing. The lack of negative feedback also enhances the belief that statical testing is useful in enhancing the down stream testing results.



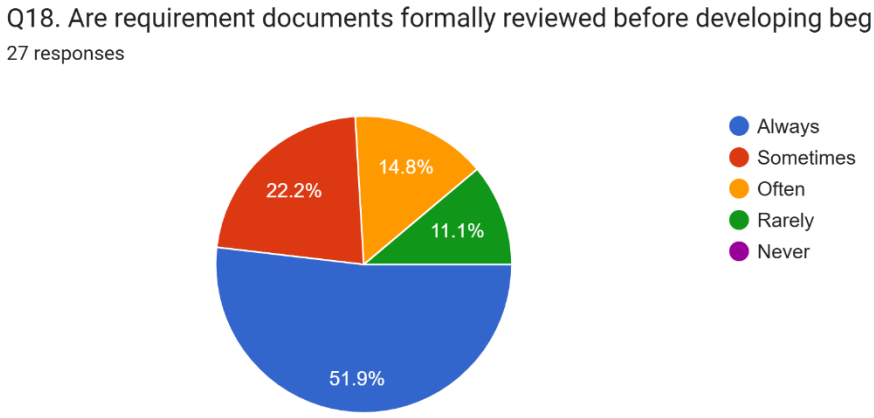
Q17. How would you rate the overall effectiveness of static testing in your projects?

The high score of 13 respondents (48.1%) rated it as Good by the largest group. This was prompted by Average with 9 respondents (33.3%), Excellent with 3 respondents (11.1%), and Poor with 2 respondents (7.4%). This is an indication that the overall impression of the concept of static testing is the positive one though not yet on the level of outstanding functioning in most organizations. The prevalence of the Good and the Average categories also suggests that the respondents acknowledge that there is something good in the practice of the static testing, but also that there is room to improve the practice and make it successful.



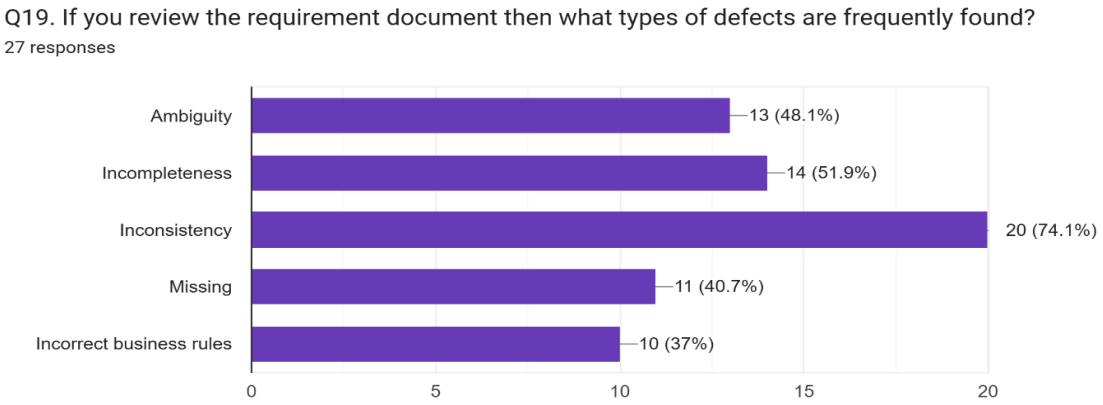
Q18. Are requirement documents formally reviewed before development begins in your project?

The replies show that the practice of requirement review is practiced to an extent of significance. Always: 14 respondents (51.9) were selected to respond to the question: Always. Sometimes: 6 respondents (22.2) were selected to answer the question: Sometimes. Often: 4 respondents (14.8) were selected to respond to the question: Often. Rarely: 3 respondents (11.1) were selected to answer the question: Rarely. None of the respondents chose Never. This implies that the vast majority of organizations do have some sort of requirement review prior to the start of development, even though the rate varies. It is also a good sign that the percentage of the responses in the Always category is relatively high, as one of the most significant types of the static testing is the validation of the requirements at the very beginning.



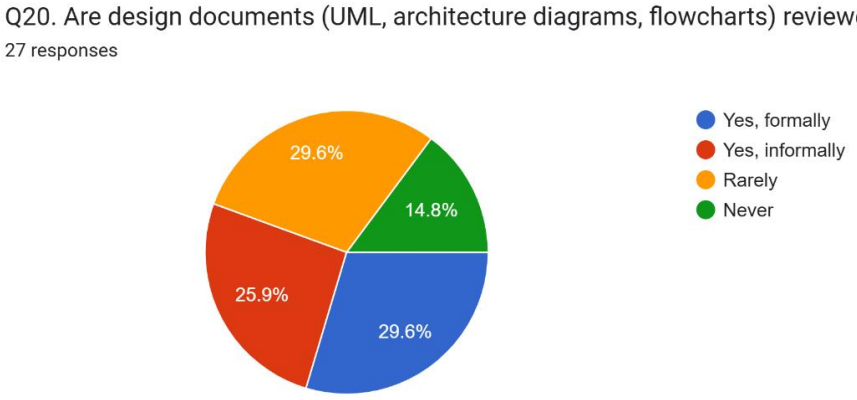
Q19. If you review the requirement document, what types of defects are frequently found?

This was a question of multiple responses. Inconsistency was the most common kind of defect, chosen by 20 participants (74.1%). This was then followed by Incompleteness with 14 respondents (51.9%), Ambiguity with 13 respondents (48.1%), Missing items with 11 respondents (40.7%), and Incorrect business rules with 10 respondents (37.0%). These results suggest that requirement reviews are particularly helpful in revealing quality issues in specifications prior to the commencement of implementation. The significance of structured requirement analysis in minimizing misunderstanding and downstream rework are emphasized by the prevalence of inconsistency, incompleteness, and ambiguity.



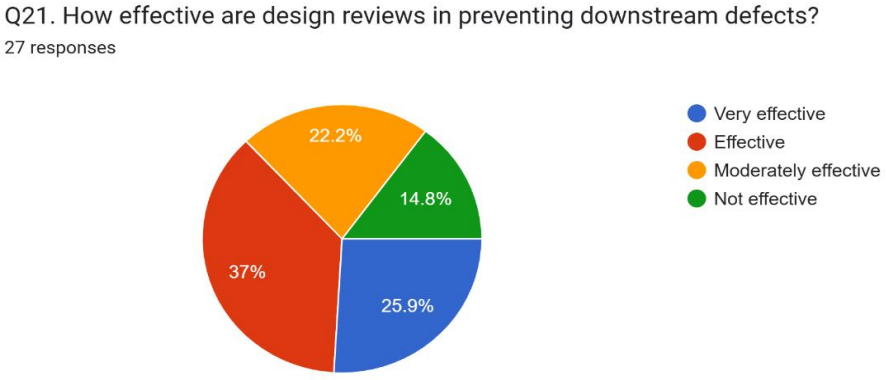
Q20. Are design documents (UML, architecture diagrams, flowcharts) reviewed statically?

The responses reflect a moderate degree of design review practice. Yes, 8 respondents (29.6%) each formally and Rarely chose both. Yes, informally was chosen by 7 respondents (25.9%), and Never was chosen by 4 respondents (14.8%). This implies design-level static testing exists, but is yet to be institutionalized consistently. Although a significant number of respondents reported having formal or informal reviews of design artifacts, the equally large number of respondents who reported having rarely conducted formal or informal reviews of design artifacts indicates that formal or informal reviews of design artifacts are not always done.



Q21. How effective are design reviews in preventing downstream defects?

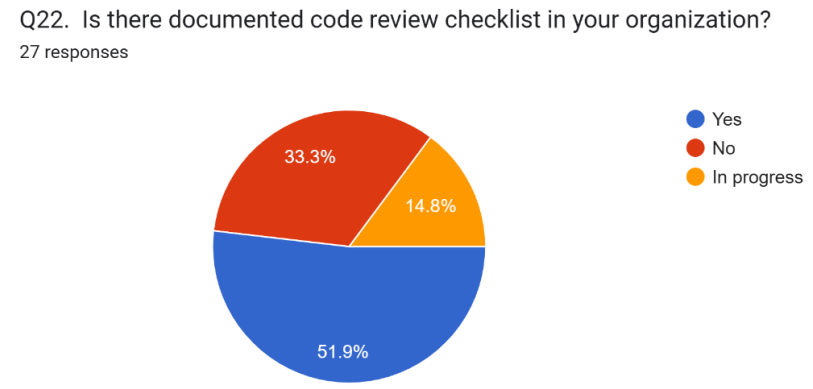
These answers reveal that the overall attitude towards design review effectiveness is rather positive. Effective was chosen by 10 respondents (37.0%), Very effective by 7 respondents (25.9%), Moderately effective by 6 respondents (22.2%), and Not effective by 4 respondents (14.8%). All in all, the findings indicate that the majority of the respondents perceive design reviews as helpful in eliminating subsequent defects, but the perception of design reviews is not overwhelming. This can be explained by the fact that the design reviews are not yet uniformly practiced by all firms, although many of the respondents recognize their importance.



Q22. Is there a documented code review checklist in your organization?

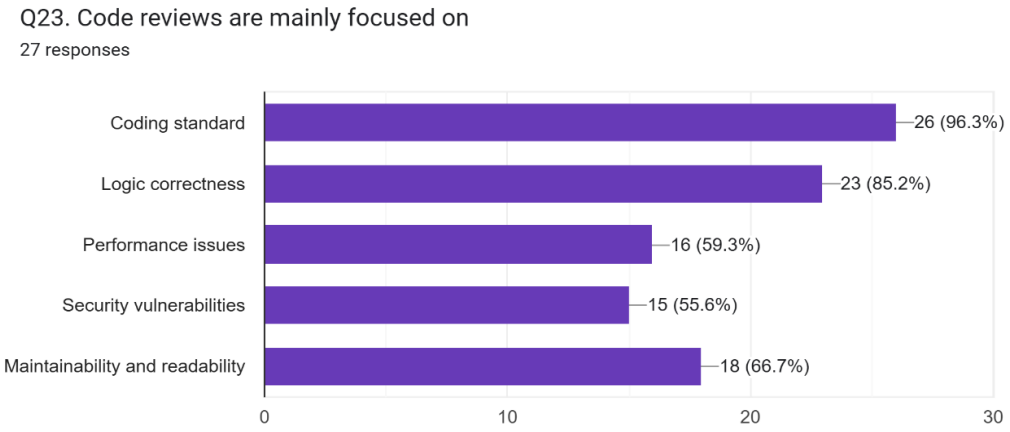
The results show that a slightly higher proportion of the participants expressed that they had a documented code review checklist in their organizations. Yes was selected by 14 respondents (51.9%), No by 9 respondents (33.3%), and In progress by 4 respondents (14.8%). The implication of this discovery is that even though most organizations already appreciate the fact

that structured review guidance is an important element of their operations, many organizations are yet to adopt a formalized checklist. The fact that there are organizations wherein the development of checklists is still in the infantile stage further supports the idea that code review standardization is still in the infancy stage. A documented checklist is significant in that it aids in maintaining consistency, minimizing the possibility of missing defects, and enhancing the quality of reviews across teams.



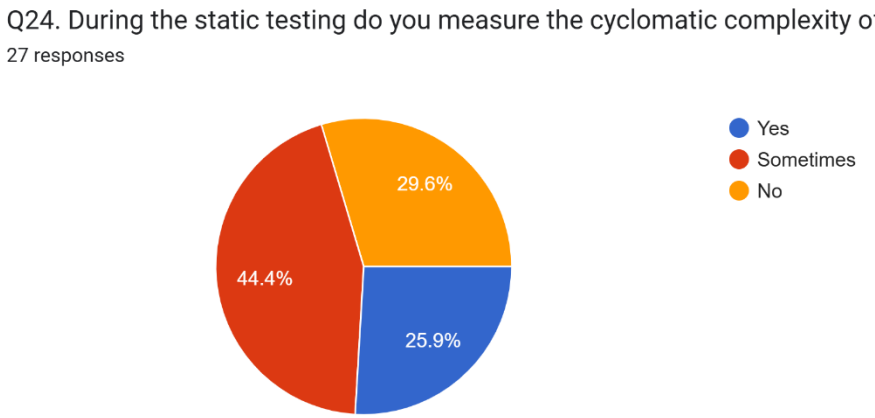
Q23. Code reviews are mainly focused on

It was a multiple-answer question, so the respondents could select over one review focus area. The outcomes indicate that Coding standard was the best-chosen item, with 26 respondents (96.3) having chosen it. It was followed by Logic correctness with 23 respondents (85.2%), Maintainability and readability with 18 respondents (66.7%), Performance issues with 16 respondents (59.3%), and Security vulnerabilities with 15 respondents (55.6%). These findings mean that the code reviews in the studied organizations are highly focused on the correctness in compliance with coding conventions and the correctness of the logic. Simultaneously, the significant portion of respondents also takes maintainability, performance, and security into account during the review activities. This is an indication that code review is not considered as a mere style-checking exercise, but a wider quality assurance activity as well. Nevertheless, the especially high level of emphasis on coding standards may suggest that not all reviews are so focused on the superficiality of the conformity criteria as to be any more focused on the deeper architectural or business-logic issues.



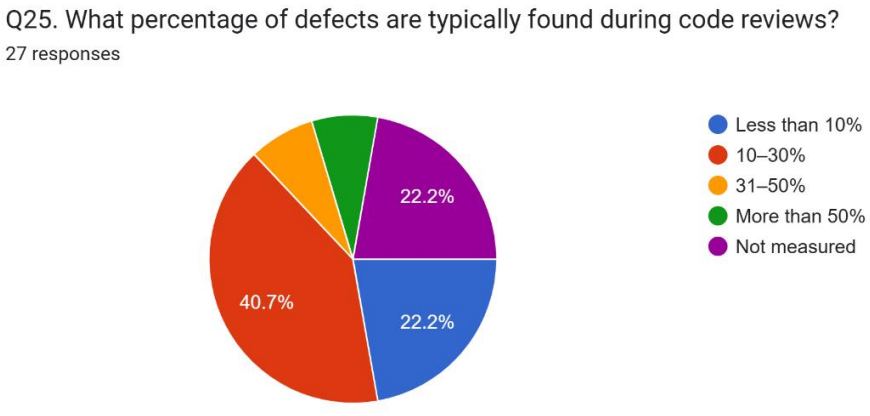
Q24. During static testing do you measure the cyclomatic complexity of the codes?

The results show that the measurement of cyclomatic complexity is not always observable. Sometimes was selected by 12 respondents (44.4%), No by 8 respondents (29.6%), and Yes by 7 respondents (25.9%). Such a distribution indicates that measuring complexity is partially incorporated into the practices of performing the static test. The fact that the most significant group of respondents had chosen the answer sometimes suggests that the given option is used irregularly, as opposed to being used systematically. Because the cyclomatic complexity is a valuable metric of the maintainability of code and difficulty of testing, the sparse and unstable use of the metric can limit the capacity of organizations to detect overly complex code at early stages.



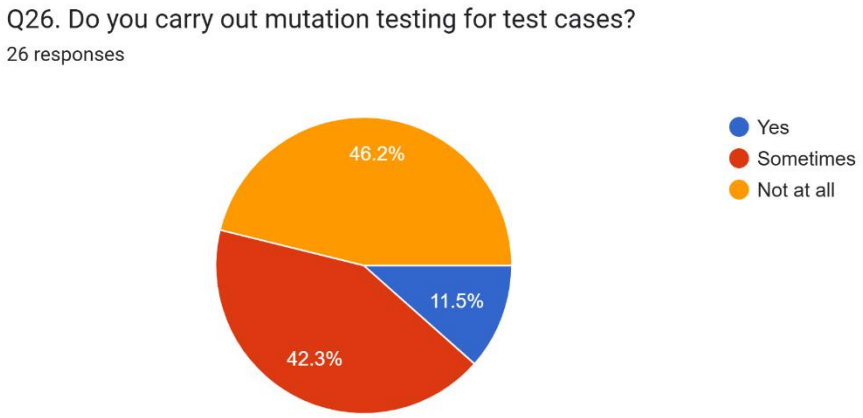
Q25. What percentage of defects are typically found during code reviews?

The responses show that the most common estimate of defects found during code reviews was 10–30%, selected by 11 respondents (40.7%). Less than 10% and Not measured were chosen by 6 respondents (22.2%). It was chosen by 2 respondents (7.4%) each in 315050, and 50More than 50. These results indicate that in most organizations, code review is viewed as the detection of a moderate percentage of all defects and not a very large percentage. Simultaneously, the fact that a relatively large number of respondents chose to answer the question with the option of Not measured suggests that a significant number of organizations do not formally set the contribution of code reviews to the detection of defects. This non-measurable nature of it may render it challenging to firms to assess the actual effectiveness of their review process, and systematically improve it.



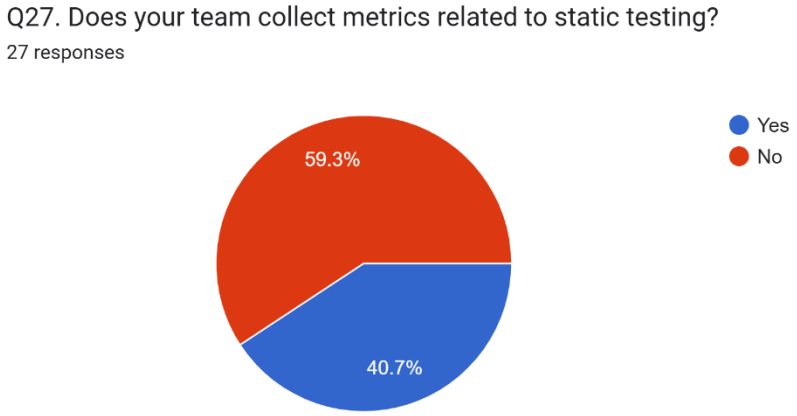
Q26. Do you carry out mutation testing for test cases?

The answers to this question were 26 and not 27. The most common ones were Not at all (12 respondents, 46.2%), Sometimes (11 respondents, 42.3%), and Yes (3 respondents, 11.5%). The findings make it apparent that mutation testing is not the norm among the surveyed organizations. A very small proportion of those surveyed reported regular use of it, with most not using it at all or occasionally. This implies that the relatively low adoption level of more advanced and sophisticated or quality-based testing practices, still remains relatively low. The reason that mutation testing is not used extensively may be due to lack of expertise, unavailability of tooling, time, or inadequate organizational focus on use of advanced quality assurance techniques.



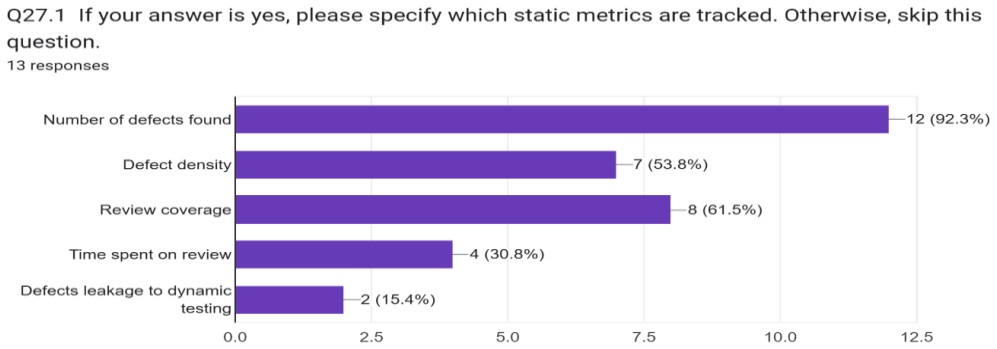
Q27. Does your team collect metrics related to static testing?

The results indicate that the set of metrics of statical testing is not yet a standard. None of the respondents (59.3% in total) chose No and 16 respondents (40.7% in total) chose Yes. This is an indication that the majority of teams are not yet practicing a measurement-based approach to static testing. Because metrics are significant in measuring performance, identifying areas of improvement and justifying investments in processes, the lack of the collection of metrics can impair the capacity of the organizations to approach the concept of the systematic and evidence-based approach to managing the concept of the process of static testing.



Q27.1 If yes, which static metrics are tracked?

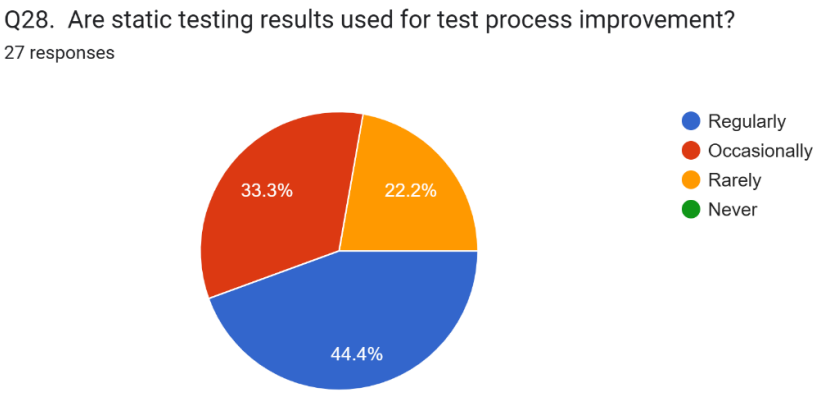
It was a multiple-response question that was answered by 13 respondents. Number of defects found was chosen as the most commonly followed measure by 12 participants (92.3%). This was preceded by Review coverage where there were 8 respondents (61.5%), Defect density where there were 7 respondents (53.8%), Time spent on review where there were 4 respondents (30.8%), and Defects leakage to dynamic testing where there were 2 respondents (15.4%). The preponderance of the number of defects found, means that most of the teams that employ metrics are mostly interested in the number of direct defects found. Measures like review coverage, defect density are also used by a moderate number of teams, and this implies that some are aware of broader quality measurement. Nevertheless, the relatively low utilization of measures like review time and defect leakage suggest that process efficiency and downstream impact are not as commonly monitored. In general, the use of metrics can be seen to be oriented towards the reporting of defects in the immediate future rather than in the overall process



assessment.

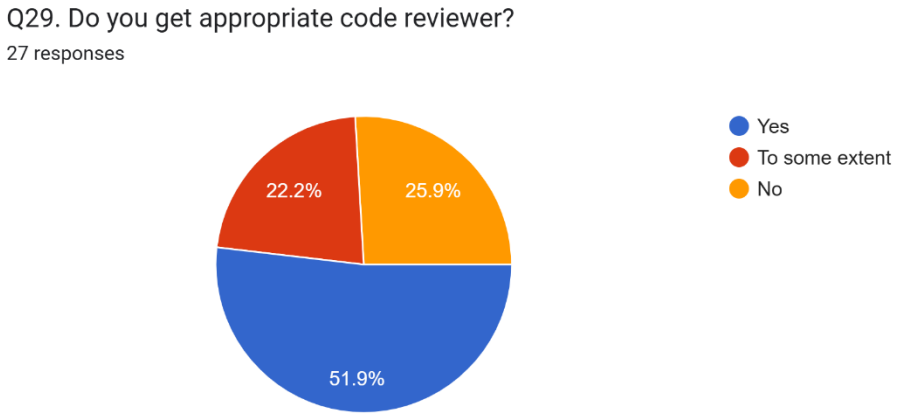
Q28. Are static testing results used for test process improvement?

The answers have shown that improvement of many organizations, although not always uniformly, is based on the static testing results. By 12 respondents (44.4%), Occasionally by 9 respondents (33.3%), and Rarely by 6 respondents (22.2%), 12 respondents (44.4%), Occasionally 9 respondents (33.3%), and Rarely 6 respondents (22.2%), was selected. None of the respondents chose Never. These results indicate that the organizations sampled tend to acknowledge the relevance of the results of the statistic tests in enhancing the testing process. Nevertheless, the fact that less than half noted that they were regularly doing this, shows that the practice is not as ingrained in all teams as it should be. Most organizations seem to apply the results of a static testing in an ad hoc or occasional fashion, and not as a part of a continuous improvement structure.



Q29. Do you get appropriate code reviewer?

The responses depict a fairly optimistic outlook when it comes to the availability of reviewers. Yes was selected by 14 respondents (51.9%), No by 7 respondents (25.9%), and To some extent by 6 respondents (22.2%). This indicates that slightly more than half of the respondents think that they have access to appropriate code reviewers, and that a large minority are experiencing partial or complete difficulty in this area. The results suggest that the suitability of reviewers is a yet significant issue of most organizations. To ensure successful static testing, it is necessary to have appropriate reviewers since such competence depends on technical abilities, familiarity with the domain, and familiarity with the codebase.



Q30. Please write what challenges you face in static testing.

This was a question on the open-ended question. Thematic review of the responses reveals that the most frequent challenges may be arranged into a few repetitive categories. Firstly, there were many mentions of time constraints and tight deadlines, with many respondents noting that statical testing is regularly rushed, delayed or even skipped as a result of delivery pressure. Second, incompleteness, ambiguity, or poor documentation of the requirements became another significant problem, and it was hard to review the artifacts effectively at an early stage. Third, respondents identified a lack of availability of reviewers, lack of expertise, and lack of domain knowledge, which may lead to lower quality of review outcomes. The lack of standardized review processes, such as inconsistent review standards, lack of formalized procedures, and problems maintaining documentation was another theme. There are also many respondents who cited tool-related issues, particularly false positives, warning fatigue, complex configuration, slow scans in large codebases, and the inability of the tools used in the tooling process to identify defects or business logic specific to the business that was the focus of the tooling process. Moreover, some of the responses referred to the lack of communication among the team, the unwillingness to receive feedback on the part of the developers, and the inability to maintain the quality of reviews in more than one repository or in projects with a rapid changeover. In general, the open-ended questions indicate that the difficulties of the static testing are both technical and organizational. They not only depend on the availability of tools, but they also go hand in hand with the pace of the project, the maturity of the process, the quality of documentation, and the teamwork.

Q31. In your opinion, what steps should be taken to improve static testing?

This was an open-ended question, therefore, respondents gave their suggestions in their own words. The responses indicate that the proposed improvements may be characterized into a few

key themes. The most recommended area of improvement was that there was a need to have more automation in the process of performing a static test. Most of the respondents suggested that automated static analysis should be integrated into the CI/CD pipeline, and pre-commit checks, automated security scans, and AI-based analytical pipelines should be used. The other significant theme was the significance of systematic review procedures. The respondents pointed out the use of review checklists, adherence to a formal review process, standardization of manual reviews, and strict practices of SDLC. The third important suggestion was that there should be clear and complete requirements and documentation. Another feature that respondents emphasized was that of training and team culture. Other recommended actions involved test-based module testing, parallel review activities, shift-left practices, multiple team member involvement, and focus human review activities on business logic and performance and accessibility rather than syntax or formatting.

Comments on Interview

4.2.1 Challenges in Static Testing

The answers to Q30 can be used to answer many of the trends that were evident in the previous survey questions. It was demonstrated that static testing was perceived to be a good idea, yet in reality it is only partially implemented, not formally defined, and lacking strong support by tools or metrics. The qualitative results of this question contribute to the explanation of the causes of such a situation. Time pressure is the most significant explanation. The idea of using static testing is commonly perceived to be helpful, yet respondents repeatedly stated that they found it challenging to implement it comprehensively due to deadlines, workload, and business urgency. Effectiveness is also diminished by weak input quality (particularly the lack of good requirements). Human beings and collaboration problems, constraints of tools and lack of full maturity of processes further explain why the application of the concept of static testing is not often done to its full potential.

4.2.2 Suggested Steps to Improve Static Testing

Responses to Q31 are also of particular significance since they will help convert the challenges identified in the process of the static testing into the improvement strategies that can be implemented. The heavy weight on automation indicates that respondents view manual static testing as a stand-alone approach that is inadequate in most real-world settings. Repeated suggestions of standardized review processes and checklists suggests that respondents think that process discipline is one of the areas that should be improved. The emphasis on improved requirements and documentation underlines a key theme in previous parts of the survey. The emphasis placed on training and team work demonstrates that static testing is not just a technical or procedural issue. Combined, the Q30 responses indicate that a multi-dimensional approach to improving the state of the art in static testing is to address all of these dimensions simultaneously.

4.3 Findings and Major Problems Identified

4.3.1 Key Findings

The results of the survey indicate that the concept of static testing is well-known among the Bangladeshi software professionals, although it is applied unequally between organizations and projects. The self reported familiarity with the respondents was mostly high, with the majority of the participants being able to match the correct associations of the static testing with the activities such as code review, requirement review, inspections and walkthroughs. But the fact that only a minority of people associated the concept of static testing with automated tools of performing a static analysis indicates that in Bangladesh the concept of static testing remains firmly linked to the concept of manual review. Although this limitation exists, the perceived benefits were evident: participants repeatedly noted the early detection of defects as a key benefit, as well as the enhanced quality of code, enhanced maintainability and reduced rework as the core ones. The other important finding is that in Bangladeshi firms, the focus of static testing is on coding stage. Although most of the respondents gave an affirmative response to the question of them performing a code review, significantly fewer of them admitted to practicing a static test as they thought about requirements and design. This is significant since the errors detected during the design stage are a thousand times less expensive to rectify than those detected after deployment. The fact that complexity measurement, mutation testing and defect tracking metrics have been barely used further points out the immature nature of process maturity. A smaller proportion of respondents (approximately half of those surveyed) stated they had a documented code review checklist and only a smaller proportion of respondents had a systematic measure of cyclomatic complexity or even a collected body of quantitative review data. In the absence of such structures, it is hard to achieve consistency or measure improvement. The survey also indicates that in Bangladeshi companies, still, the process of the static testing is mainly manual. Informal reviews and peer reviews were prevalent, whereas the use of the tools of the static analysis was very low. Such dependence on human effort renders the process susceptible to the availability of reviewers, their level of skills and subjective evaluation. Furthermore, the survey indicated that a significant number of the respondents inaccurately described dynamic testing tools like Selenium and JMeter as static testing tools. This conflation indicates that there are still knowledge gaps as to the difference between the techniques of the past and those of the present and that there is a need to train and sensitize programmers.

One last conclusion is that of the organizational support. Most of the respondents answered that only partial conduction of the static testing is done; they cited tight schedules, lack of trained reviewers and incomplete requirements as reasons behind non-conducting of the static testing or rushing the conduction of the test. Nonetheless, the survey respondents rated design and requirement reviews as fairly effective and they supported the use of results of the static testing to enhance future testing procedures. Overall, the most important findings suggest that Bangladeshi software companies recognize the importance of formal testing but have an uneven approach to it, and their use of it is inadequately automated, with poor metrics and little early phase focus.

4.3.2 Major Problems Identified

The survey and literature provided several systemic issues. The most evident concern is the inconsistent adoption: the practice of static testing is usually seen as optional, subject to project timelines and customer requirements instead of as a quality standard practice. This discrepancy decreases the preventive value of early reviews. The second issue is the excessive dependence on the manual review. Due to the infrequent use of automated static analysis by few organizations, reviews rely on human skill and availability. Such a dependency is associated with slower feedback loops, challenges in scaling to large codebases, and inconsistency in the quality of the review. Combining automated analysis with human review results are better than either method alone, as emphasized by international studies.

The third issue is ineffective testing in the initial phases. Although it is cost-effective to identify defects in requirements and design, most companies delay to do so until they code or even later. This delay can be partly attributed to inadequate documentation and unclear requirements, which complicate early reviews. The survey data also suggest that there are no recorded review procedures, infrequent checklists, and insufficient training. Reviews conducted informally may fail to identify any important problems or degenerate into discussions of style.

The other problem is a lack of tool adoption and configuration. Confusion over what a static testing tool is and false positives, complexity in configuration, slow scans are reasons not to use tools. International studies confirm that even with proper settings, the precision of the static analyzers can be very different and may also issue excessive warnings, yet they are still useful as long as they are properly configured and supplemented with human judgement. The low rate of tool utilization in Bangladesh is probably because of the cost sensitivity, lack of expertise and the belief that manual review is sufficient.

Lastly, time limits are imposed by resource constraints in terms of tight deadlines, lack of qualified experienced reviewers and low tester to developer ratios. Other factors cited by the respondents included resistance by the developers, roles-to-role communication, and management non-support. These difficulties are also reflective of the general findings of the literature on Bangladeshi SQAT studies, which have identified time pressure, lack of process discipline and insufficient training as enduring challenges.

4.4 Recommendations

To enhance the use of software firms as the more structured and proactive approach to improving the practice of software firms in Bangladesh, software firms should take a more organized and proactive approach that is considered internationally best practice, and that takes into account the local constraints. The following recommendations are related to organizational, technical and human factors:

1.Make static testing part of the SDLC. Management must enforce a practice of static testing at significant points in the process, such as requirements, design and coding, and not leave it to individual teams. Early reviews decrease downstream cost and risk.

2.Use standard processes and checklists. Document reviews using checklists specific to coding guidelines, security policies and domain necessities. Consistency enhances the coverage and helps in the development of mutual understanding.

3.Assimilate automatized tools of statical analysis. Continuous integration pipelines may include open source tools (e.g., ESLint, SonarQube, PMD) to provide immediate feedback and to spot issues systematically. The teams may recommend to mitigate false positives, rule sets should be modified and multiple tools should be combined as well as human review should be focused on high risk areas, which is recommended in the international literature.

4.Move to the left and enhance documentation. Promoting the concept of reviewing requirements and designing documents to an extent before coding. Comprehensive documentation allows an early review to be meaningful, and reduces ambiguity. The lightweight inspection models which include the QIM can be used to support the resource constrained teams.

5.Invest in training and cross functional teamwork. Conduct continual training on how to perform static testing, how to configure and review tools to the developers, testers and project managers. Engage business analysts, and product owners in requirement and design reviews to verify that domain knowledge is covered.

6.Set time and resources aside to conduct reviews. Project plans must have specific time set aside to do the static testing and organizations must have competent reviewers available. The management should also appreciate that, instead of being a discretionary endeavor, management should recognize that static testing is a cost saving investment.

7.Collect and use metrics. Monitor defect density, review coverage, time to review and defect leaking to dynamic testing. These measures allow teams to determine the efficiency of the static testing, determine bottlenecks and promote continuous improvement.

8.Promote a culture of quality. Foster free feedback, applaud defect prevention and stress that reviews are not punitive. Support of leaders is crucial in order to make sure that the practice of static testing becomes embedded into the organizational practice.

The following recommendations will enable Bangladeshi software companies to utilize the maximum benefits of static testing, minimize the use of manual practices and conform to the international standards. The literature highlights the fact that static testing is a proactive approach that promotes collaboration, increases maintainability and enhances software security. By moving left, using the right tools and creating people and processes, organizations can develop quality software in the most efficient and competitive

Chapter 5

Conclusion

5.1 Introduction

This chapter presents the conclusion of the study on static testing practices in software engineering with a focus on Bangladesh. The purpose of this research was to understand the current use of static testing, identify its benefits and limitations, and examine the major challenges organizations face while applying static testing activities in software development. The chapter also summarizes the main research outcomes, presents brief recommendations, discusses the constraints of the study, and suggests possible directions for future work.

The findings of this study indicate that static testing plays an important role in improving software quality by identifying defects at an early stage of the software development life cycle. Literature and survey findings both suggest that static testing helps reduce defect leakage, lowers rework cost, improves documentation quality, and supports overall software quality assurance. At the same time, the study also reveals that the adoption of static testing practices in Bangladesh is still not fully mature, especially in smaller organizations where budget, skilled personnel, and structured processes remain limited. These observations are consistent with prior Bangladesh-focused studies that report gaps in formal testing teams, process maturity, training, and early quality assurance practices. [1][3]

5.2 Research Outcome and Recommendation in Brief

The study produced several important outcomes. First, it confirms that static testing is an essential practice for improving software quality because it allows faults to be detected before code execution. Activities such as reviews, inspections, walkthroughs, and static analysis help identify problems in requirements, design, and source code at an early stage. This early detection reduces the cost of fixing defects later in development and contributes to better-quality software products.

Second, the study found that although static testing is recognized as important, its practical application is inconsistent. Many organizations still rely more heavily on dynamic testing, while static testing activities often receive less attention. In Bangladesh, previous survey evidence shows that testing is widely practiced, but not always in a structured and mature way, and some firms still lack dedicated test teams or formal quality models .[7][8] The same survey also highlights common challenges such as maintaining quality within limited budget and time, lack of skilled personnel, difficulty in early testing, and problems understanding requirements .[8]

Third, the survey findings used in this thesis show that organizations understand the value of defect prevention, review practices, and documentation-based checking, but these are not always applied systematically. Static testing tends to be more effective where teams have clearer requirements, better review culture, checklists, proper time allocation, and trained reviewers. Where these conditions are absent, static testing becomes irregular and less effective.

Based on these outcomes, several brief recommendations can be made. Organizations should give greater importance to static testing as part of their regular software development practice rather than treating it as optional work. Review activities should begin from the requirements and design stages so that defects can be prevented before implementation. Companies should provide training on review techniques, static analysis tools, and defect reporting so that team members can perform static testing more effectively. Software firms should also develop structured review procedures, use checklists, maintain documentation and traceability, and encourage collaboration between developers, testers, and analysts. In addition, academic institutions and training centers should strengthen software testing and quality assurance content in their curriculum, since previous Bangladesh-based evidence shows the present curriculum is often seen as inadequate for preparing potential testers . [8] Greater emphasis on process maturity, awareness programs, and training infrastructure has also been recommended in prior Bangladesh studies .[7][8]

5.3 Constraints

This study has several constraints that should be acknowledged. First, the research was based on a limited number of survey responses and available literature. Although the survey findings provide useful insight, they may not fully represent all software organizations in Bangladesh, especially very small firms or companies outside major urban centers. Similar limitations have been noted in broader Bangladesh software engineering studies, where smaller and rural firms may be underrepresented .[8]

Second, the study focused mainly on perceptions, reported practices, and literature findings rather than direct observation of industrial projects. As a result, some findings reflect the opinions and experiences of respondents rather than measured performance data from real project environments.

Third, the research concentrated on static testing practices and did not perform a deep technical comparison of specific static analysis tools in Bangladeshi companies. Therefore, the study identifies the importance and challenges of static testing but does not experimentally evaluate which tools are most effective in a local industrial context.

Fourth, time and resource limitations restricted the scope of the study. A larger sample, more company case studies, and interviews with senior quality assurance professionals could have provided a richer and more comprehensive picture.

Finally, the study mainly examined the current state and general challenges of static testing. It did not explore in full detail how differences in company size, project type, development methodology, or domain-specific requirements influence the adoption of static testing.

5.4 Possible Future Work

Several directions can be considered for future research. A larger empirical study can be conducted with more software companies from different regions of Bangladesh to obtain a broader picture of static testing practices. Such a study could compare startups, mid-sized companies, outsourcing firms, and larger software organizations to understand how organizational size and business type affect static testing adoption.

Future research may also focus on detailed case studies of companies that successfully use static testing. This would help identify best practices, management support strategies, review workflows, and tool adoption models that others can follow. Another useful direction would be to evaluate specific static analysis tools in real Bangladeshi software projects and compare their effectiveness, usability, cost, and suitability for local industrial needs.

Further work can also investigate the relationship between static testing and agile development in Bangladesh, since earlier literature shows that agile teams often emphasize dynamic testing more than static techniques, even though static practices can improve early defect detection. Research can also explore how static testing can be integrated with DevOps, CI/CD pipelines, and secure software development practices.

In addition, future studies may examine the impact of academic curriculum, professional certification, and industry training on the readiness of graduates and practitioners to perform static testing effectively. Since lack of training and skills remains a major challenge, research in this area could help bridge the gap between education and industry demand. Prior Bangladesh-focused work also suggests the need for more detailed studies on automated testing, startup-focused software quality practices, and requirements engineering challenges, all of which connect closely with the improvement of static testing culture. [8]

In summary, this study shows that static testing has strong potential to improve software quality in Bangladesh, but its wider success depends on awareness, training, structured practice, and organizational commitment. Future research can build on this foundation by expanding empirical evidence, evaluating practical methods, and proposing models that support broader and more effective adoption of static testing in the Bangladeshi software industry.

References

[1] D. Nikolić, D. Stefanović, D. Dakić, S. Sladojević, and S. Ristić, "Analysis of the Tools for Static Code Analysis," in Proc. 20th Int. Symp. INFOTEH-JAHORINA, 2021.

[2] T. R. G. Nair, V. Suma, and P. K. Tiwari, "Analysis of Test Efficiency during Software Development Process," in Proc. 2nd Annu. Int. Conf. Software Engineering and Applications (SEA 2011), 2011.

[3] A. K. Joshy, X. Chen, B. Steenhoek, and W. Le, "Validating Static Warnings via Testing Code Fragments," in Proc. 30th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA '21), Jul. 2021, doi: 10.1145/3460319.3464832.

[4] E. A. AlOmar, S. A. AlOmar, and M. W. Mkaouer, "On the Use of Static Analysis to Engage Students with Software Quality Improvement: An Experience with PMD," arXiv preprint arXiv:2302.05554, 2023.

[5] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "An Empirical Study of Static Analysis Tools for Secure Code Review," arXiv preprint arXiv:2407.12241, 2024.

[6] M. A. Kashem and R. J. Bhuiyan, "A cost reduction method in SQA by using inspection and static analysis tool," IOSR Journal of Computer Engineering, vol. 9, no. 1, pp. 26-29, Jan.-Feb. 2013.

[7] A. M. Kale, V. V. Bandal, and K. Chaudhari, "A Review Paper on Software Testing," International Research Journal of Engineering and Technology (IRJET), vol. 6, no. 1, Jan. 2019.

[8] S. M. A. R. Bhuiyan, M. S. Rahim, A. Z. M. E. Chowdhury, and M. H. Hasan, "A Survey of Software Quality Assurance and Testing Practices and Challenges in Bangladesh," International Journal of Computer Applications, vol. 180, no. 39, May 2018, doi: 10.5120/ijca2018917063.

[9] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," Journal of Systems and Software, vol. 198, Art. no. 111575, 2023, doi: 10.1016/j.jss.2022.111575.

[10] A. Oluwaferanmi, "Adopting Static and Dynamic Application Security Testing (SAST & DAST) for Continuous Vulnerability Detection Across the SDLC," unpublished manuscript, Jul. 7, 2025.

[11] N. Sultana, M. Syeed, and K. Fatema, "An Empirical Investigation on The Quality Assurance Practices in Software Industries: Bangladesh Perspective," International Journal of Software Engineering and Computer Systems, vol. 6, no. 2, pp. 1-10, 2020, doi: 10.15282/ijsecs.6.2.2020.1.0070.

[12] S. R. Yusuff, A. O. Bajeh, T. O. Aro, and K. S. Adewole, "Improving the Accuracy of Static Source Code Based Software Change Impact Analysis Through Hybrid Techniques: A Review," International Journal of Software Engineering and Computer Systems, vol. 7, no. 1, pp. 57-66, 2021, doi: 10.15282/ijsecs.7.1.2021.6.0082.

[13] M. Gleirscher, D. Golubitskiy, M. Irlbeck, and S. Wagner, "Introduction of Static Quality Analysis in Small and Medium-Sized Software Enterprises: Experiences from Technology Transfer," Software Quality Journal, accepted manuscript, 2016.

[14] H. S. Dar, M. U. Ali, and J. A. Shaikh, "Influence of Static Testing in Agile Development: A Case From Pakistan Software Market," Sci.Int. (Lahore), vol. 26, no. 5, pp. 2019-2022, 2014.

[15] M. A. Islam, M. S. S. Khan, A. Al Mahi, H. R. Ishfar, M. H. Bashar, M. F. Bakhtiar, and A. Kalam, "Software Engineering in Bangladesh: Bridging Local Practices with Global Standard," North American Academic Research, vol. 7, no. 12, pp. 85-97, Dec. 2024, doi: 10.5281/zenodo.14504080.

[16] S. G. Tetteh, "Software Testing Techniques and Levels in Software Development," ESP International Journal of Advancements in Computational Technology, vol. 2, no. 1, pp. 10-19, Feb. 2024, doi: 10.56472/25838628/IJACT-V2I1P102.

[17] A. Ghorai, "Static Testing in Software Engineering - Reducing Defect Leakage," International Journal of Science and Research (IJSR), vol. 7, no. 5, May 2018, doi: 10.21275/SR24509123132.

[18] A. Tymoshchuk, "The Role of Testing in Software Security Assurance," International Journal of Engineering and Computer Science, vol. 14, no. 3, pp. 27014-27021, Mar. 2025, doi: 10.18535/ijecs/v14i03.5049.

Appendix A—Survey Form



Survey on software static testing

Declaration :Your answers to this survey will be considered as confidential and will be used only for research purposes.

Section A: Interviewees' background

Your Name……Firm's Name: ……..

1. What is your current role?

☐ Requirement Engineer ☐ Developer ☐ Q/A Test ☐ Project Manager ☐Other_____

2. How many years of experience do you have in software development or IT? ……..

3. What types of projects does your software firm

☐ For localcustomers ☐ Outsourcing ☐ Others (please specify) …

Section B: Awareness and understanding of static testing

1. How familiar are you with the concept of static testing?

☐ Very familiar ☐ Familiar ☐ Somewhat familiar ☐ Not familiar

2. Which of the following do you consider part of the static testing? (Select all that apply)

☐ Code review ☐ Walkthrough ☐ Inspection ☐ Requirement Review ☐ Static code analysis using tools ☐ Compilation only

3. At which stages of the software life cycle (SDLC) is static testing performed in your projects?

☐ Requirement phase ☐ Design phase ☐ Coding phase ☐ Testing Phase ☐ Not practiced

Section C: Static testing practice

1. Does your organization formally practice static testing?

☐ Yes ☐ No ☐ Partially

2. What static testing techniques are used in your projects? (select which are applied)

☐ Informal review ☐Formal inspections ☐Peer review ☐Walkthroughs ☐ Automated static analysis tools

3. Who usually participates in static testing activities?

☐ Developers ☐ Testers ☐ Business Analysts ☐ Project Managers ☐ All stakeholders

4. How often are code reviews conducted?

☐ For every module when finished ☐ Weekly ☐ Before Delivery ☐ None

Section D: Testing Tools used

1. Do you use any static testing tools?

☐ Yes ☐ No

If yes, then mention the name of the tool ,,,,,,,,,,,,,,,,

2. How effective are those tools (if you use the tool)

☐ Very effective ☐ Effective ☐ Moderately effective ☐ Not effective

Section E: Benefits and effectiveness

1. What benefits have you observed from static testing?

☐ Early defect detection ☐ Reduced testing cost ☐ Improved code quality ☐ Better maintainability ☐ Reduces rework ☐ No noticeable benefits

2. Static testing helps reduce defects found during dynamic testing.

☐ Strongly agree ☐ Agree ☐ Neutral ☐ Disagree ☐ Strongly Disagree

3. How would you rate the overall effectiveness of static testing in your projects?

☐ Excellent ☐ Good ☐ Average ☐ Poor

Section F: Requirement and Design Static Testing

1. Are the requirement documents formally reviewed before development begins in your project?

☐ Always ☐ Often ☐ Sometimes ☐ Rarely ☐ Never

2. If you review the requirement document, then what types of defects are frequently found?

☐ Ambiguity ☐ Incompleteness ☐ Inconsistency ☐ Missing ☐Incorrect business rules

3. Are design documents (UML, architecture diagrams, flowcharts) reviewed statically?

☐Yes, formally ☐Yes, informally ☐ Rarely ☐Never

4. How effective are design reviews in preventing downstream defects?

☐ Very effective ☐Effective ☐ Moderately effective ☐Not effective

Section G: Code review process

1. Is there a documented code review checklist in your organization?

☐ Yes ☐ No ☐ In progress

2. Code reviews are mainly focused on

☐ Coding standard ☐ Logic correctness ☐ Performance issues ☐Security vulnerabilities
☐ Maintainability and readability

3. During the static testing, do you measure the cyclomatic complexity of the code?

☐ Yes ☐Sometimes ☐ No

4. What percentage of defects are typically found during code reviews?

☐ Less than 10% ☐10-30% ☐ 31-59% ☐ More than 50% ☐ Not measured

5. Do you perform mutation testing on test cases?

☐ Yes ☐ Sometimes ☐ Not at all.

Section H: Metrics and measurement

1. Does your team collect metrics related to static testing?

☐ Yes ☐ No

If your answer is Yes, then what static metrics are tracked?

☐ Number of defects found ☐ Defects density ☐Review coverage ☐ Time spent on review
☐ Defects leakage to dynamic testing

2. Are static testing results used to improve the test process?

☐ Regularly ☐ Occasionally ☐ Rarely ☐ Never

Section I Challenges

1. Do you get an appropriate code reviewer?

☐Yes ☐ To some extent ☐No

2. Please write what challenges you face in static testing? Please mention below

3. In your opinion, what steps should be taken to improve the static testing?

Appendix B—Face to Face Interview

Developing Checklist

Even though the questionnaire survey was the primary data collection tool of this study, a face-to-face interview checklist was also prepared as a supplementary qualitative instrument to explore the topic of our study of the practices of the statical testing deeper. The aim of writing this checklist was to come up with a structured checklist that could be used to authenticate survey results through direct communication with software professionals, team leads, QA engineers and project managers.

The checklist was created based on the most significant themes found in the review of the literature and the design of the survey. The previous research highlights that the organizational structure, maturity of the processes, time constraints, resource limitations and the awareness of formal quality techniques influence the practice of software quality assurance in Bangladesh [1], [2]. Likewise, the significance of reviews, checklists, tool support, and management commitment is also emphasized in literature on the subject of static quality analysis and inspection-based quality assurance [2], [4], [7]]. On the basis of these findings, the interview checklist was created in such a way that any face-to-face discussion would not be digressive, inconsistent and irrelevant to the research objectives.

The checklist was organized around the following areas:

1. Organizational background

• Type and size of the organization.
• Development team structure presence of QA or testing people.
• Nature of software projects that the organization deals with.

2. Knowledge of non-dynamical testing.

• Familiarity of the participants with the concept behind the notion of static testing;

• Perceived difference between the testing when in a static state and when in a dynamic state;

• Importance of reviews and inspections in the SDLC.

3. Current practices

• Whether requirement reviews, design reviews, code reviews or inspections are done;
• Who is involved in the review activities whether there are formal checklists or guidelines in use.
• Frequency and documentation of outcome of review.

4. Tool support

- use of static analysis tools or code quality tools;
- purpose of tool usage, such as bug detection, coding standard enforcement, or security checking;
- perceived usefulness and limitations of such tools.

5. Challenges

- lack of time for review activities;
- shortage of skilled reviewers;
- poor documentation quality;
- lack of management support;
- developer resistance or overreliance on dynamic testing.

6. Improvement suggestions

- training needs for static testing;
- inclusion of checklist-based reviews in project workflow;
- better documentation and traceability practices;
- automation and tool integration;
- stronger organizational emphasis on preventive quality assurance.

This checklist-based structure was intended to improve consistency in qualitative inquiry and to reduce interviewer bias. It also reflects the established view in the literature that checklist-guided inspections and reviews can improve defect detection and support more systematic quality assurance activities [2], [4], [6].

In this thesis, the checklist is included as a methodological support instrument for future or extended qualitative validation. Its development is important because it demonstrates that the study considered not only broad survey responses but also the possibility of obtaining deeper practitioner insights through structured discussion.